PROJECT REPORT

ARM Code Optimization for Image Converting

과 목 마이크로프로세서 응용

담당 교수	이영서 교수님
전공	전자정보공학부 IT 융합
조원 (1조)	박진서 (20222971) 이건아 (20201599) 장민경 (20221616) 정재훈 (20202996)

목차

Ⅰ. 서론

1. 개요
2. 프로젝트 수행 일지

Ⅱ. 본론

1. 32-bit RGBA 이미지를 메모리에 저장하는 ARM Code
2. 이미지 변환 함수의 동작 방식 및 Block diagram
3. 3가지 이미지 변환 함수의 ARM Code
4. Memory Relocation과 이를 적용한 변환 함수 ARM Code
5. 기존 ARM Code와 최적화된 ARM code 사이의 성능 비교
6. ARM Code 추가 최적화
7. 실제 알고리즘에 기반한 결과 (이미지) 도출

Ⅲ. 결론

1. 프로젝트 수행에 대한 고찰 (Troubleshooting)
2. 프로젝트 결과와 느낀 점

Ⅰ. 서론

1. 개요

본 프로젝트는 Keil MDK를 활용하여 이미지 처리 및 변환을 수행하는 ARM 코드를 작성하고 최적화하여 성능 비교를 수행한다. 기존 이미지를 8-bit RGB 이미지, Grayscale 이미지 그리고 Binary 이미지로 변환하는 ARM 코드를 작성하고 픽셀을 메모리에 저장할 때 재배치하여 ARM 코드를 최적화한다. 마지막으로 기존 ARM 코드와 최적화된 ARM 코드의 성능 비교 분석을 통해 최적화의 효과를 확인하여 결과를 도출하고자 한다.

본 프로젝트에서 사용하는 ARM Keil MDK는 Arm Cortex-M (M23/M33 포함) 기반의 임베디드 개발을 위해 설계된 강력한 통합 개발도구이다. 임베디드 시스템 개발의 전체 프로세스 과정을 지원하는 Arm Keil MDK는 RTX RTOS를 지원하여 RTOS 기반의 애플리케이션 개발을 보다 빠르고 손쉽게 개발할 수 있도록 도와준다. TUV SUD로부터 자동차, 산업, 철도, 의료, 항공 등의 분야에 인증을 받은 FuSa RTS는 기능 안전이 필요한 소프트웨어 개발에 있어 다양한 라이브러리와 구성요소로 안전한 개발 환경을 구축할 수 있다. ARM Keil MDK를 활용할 경우, 애플리케이션 코드에만 집중할 수 있는 최적의 임베디드 개발 환경을 구축할 수 있다.

2. 프로젝트 수행일지

이름	장민경 (조장)	박진서	이건아	정재훈
모임 일자	**수행 내용**			
5 / 10	- 프로젝트 **문제 분석 및 계획 수립** - **개인 역할 분담** 1) 장민경 : 수행 일지 작성, 보고서 취합, 발표 자료 작성 2) 박진서 : 보고서 10번 항목 작성, 발표 및 발표 녹화 3) 이건아 : 보고서 1번 및 7번 항목 작성, 발표 자료 작성			

	4) 정재훈 : 보고서 3번 항목 작성, 발표 자료 작성 - 32-bit RGBA 이미지를 메모리의 연속된 주소 공간에 저장하는 함수의 ARM Code 작성 시도 (**보고서 2번 항목)**
5 / 19	- 보고서 2번 작성 완료 및 계획 수정 - 3가지 이미지 변환 함수의 동작 방식 ARM Code 역할 분담 (**보고서 3번 항목**) 공통) 8-bit RGB image 1) 장민경 : Binary image 2) 박진서 : Grayscale image 3) 이건아 : Grayscale image 4) 정재훈 : Binary image
5 / 20	- 보고서 3번 해결 방법 토의 - **역할 분담 수정** 1) 장민경 : 보고서 1, 2, 5, 7번 작성 및 취합, 발표 자료 작성 2) 박진서 : 보고서 3, 4, 10번 작성, 발표 및 발표 녹화 3) 이건아 : ARM 코드 작성, 발표 자료 작성 및 취합 4) 정재훈 : ARM 코드 작성, 발표 자료 작성 및 취합
5 / 25	- 3가지 이미지 변환 함수 ARM 코드 완성 - ARM 코드 최적화를 위한 A 제외 코드 (**Memory Relocation**) 완성 - ARM 코드 최적화 수행 - 순서도 (**Block diagram**) 완성 - 발표 자료(PPT) 구성 논의

5 / 31	- PPT 최종본 완성 및 대본 작성 - 보고서 첨삭 및 최종 수정 - 발표 영상 촬영

Ⅱ. 본론

1. 32-bit RGBA 이미지를 메모리에 저장하는 ARM Code

본 프로젝트에서는 32-bit RGBA 이미지를 메모리의 연속된 주소 공간에 저장하는 것을 ARM Code로 구현하는 것 대신 이미지 파일을 .hex 파일로 변환 후 해당 파일을 Keil MDK Terminal을 통해 Load하는 것으로 대체하였다. 이후 원활한 프로젝트 진행을 위해 이미지의 Header와 Pixel 데이터를 구분하는 것이 필요하였고 이를 ARM Code로 구현하였다.

(1) "IDAT"를 판별하여 Header와 Pixel을 분리하는 Code

```
1       AREA RESET, DATA, READONLY
2       AREA Image, CODE, READONLY
3
4       ENTRY
5       ; Initializing
6       MOV R1, #0x40000000 ; Starting Address of Input (Image)
7       MOV R2, #0x20000000 ; Address of "Memory storing HEADER" and "End of Header"
8       MOV R3, #0x30000000 ; Address of "Memory storing PIXEL" and "End of PIXEL"
9       MOV R4, #0          ; Clear register to use for loading and storing
10      MOV R5, #0
11      MOV R10, #0
12      B HEADER
13
14  HEADER_END ; Judge the end of header, "IDAT"
15      SUB R2, R2, #2 ; < Prepare to judge "IDAT" >
16      MOV R7, #0
17      LDRB R4, [R2], #-1 ; < Compare with 'A' >
18      MOV R6, R4
19      CMP R6, #0x41
20      ADDNE R7, R7, #1 ; If it is not 'A', Check at R7
21      LDRB R4, [R2], #-1 ; < Compare with 'D' >
22      MOV R5, R4
23      CMP R5, #0x44
24      ADDNE R7, R7, #1 ; If it is not 'D', Check at R7
25      LDRB R4, [R2], #-1 ; < Compare with 'I' >
26      CMP R4, #0x49
27      ADDNE R7, R7, #1 ; If it is not 'D', Check at R7
28      STRB R4, [R2, #1]! ; Restore to its original state
29      STRB R5, [R2, #1]!
30      STRB R6, [R2, #1]!
31      ADD R2, R2, #2
32       CMP R7, #0
33      MOV R5, #0 ; Reset R5 to use
34      BEQ PIXEL ; If Last 4 bits are "IDAT", Go to PIXEL
35
```

```
36 HEADER ; Store bits of header from image
37       LDRB R4, [R1]
38       STRB R10, [R1], #1
39       MOV R5, R4
40       STRB R4, [R2], #1
41       CMP R5, #0x54 ; If current bit is 'T', Check that is 'T' of "IDAT"
42       BNE HEADER
43       B HEADER_END
44
45 PIXEL ; Store bits of pixel from image
46       LDRB R4, [R1]
47       STRB R10, [R1], #1
48       STRB R4, [R3], #1
49       ADD R5, #1
50       CMP R5, #614400 ; 614400 = 960 * 160 * 4 (Bytes)
51       BNE PIXEL
52       END
```

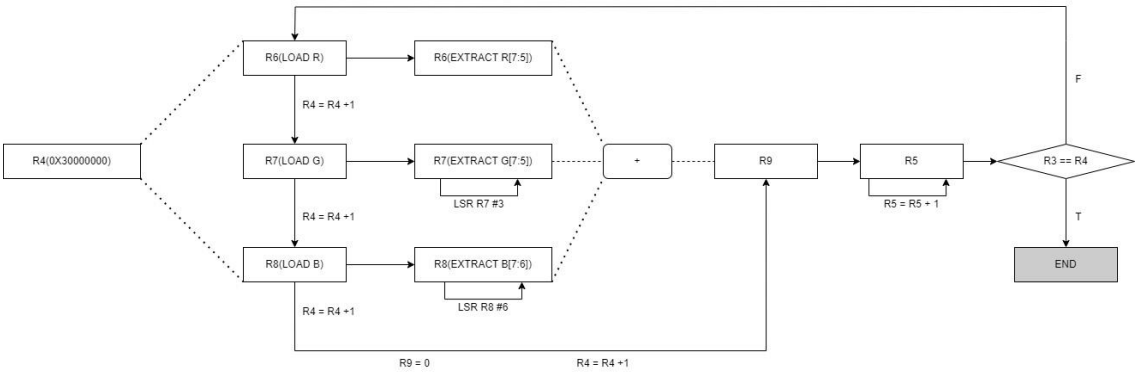
(2) Header Size를 지정하여 Header와 Pixel을 분리하는 Code

```
1       AREA     RESET, CODE, READONLY
2
3       ENTRY                       ; Linker entry point marker
4
5       ; Initialize pointers
6       LDR   R1, =0x40000000       ; Source start address for the header section
7       LDR   R2, =0x20000000       ; Destination start address for the header section
8       LDR   R3, =0x4000003E       ; End address for the header section (last byte to copy)
9       MOV   R5, #0                ; Value to clear memory with
10
11      ; header_section copy loop
12  header_section
13      LDRB  R4, [R1]              ; Load a byte from source
14      STRB  R4, [R2], #1          ; Store the byte at destination and increment destination address
15      STRB  R5, [R1], #1          ; Clear source location and increment source address
16      CMP   R1, R3                ; Compare current source address with the end address
17      BLT   header_section    ; If not reached, loop back to copy more
18
19      ; Update pointers for the header section
20      LDR   R1, =0x4000003E       ; Source start address for the pixel section
21      LDR   R2, =0x30000000       ; Destination start address for the pixel section
22      LDR   R3, =0x40126C12       ; End address for the pixel section
23
24      ; Pixel section copy loop
25  pixel_section
26      LDRB  R4, [R1]              ; Load a byte from source
27      STRB  R4, [R2], #1          ; Store the byte at destination and increment destination address
28      STRB  R5, [R1], #1          ; Clear source location and increment source address
29      CMP   R1, R3                ; Compare current source address with the end address
30      BLT   pixel_section   ; If not reached, loop back to copy more
31
32      ; Finish pixel section copy
33      CMP   R1, R3                ; Check if end of data reached
34      BNE   pixel_section         ; Continue copying if not finished
35
36  ; Finish execution
37      B     .
38
39      END
40
```

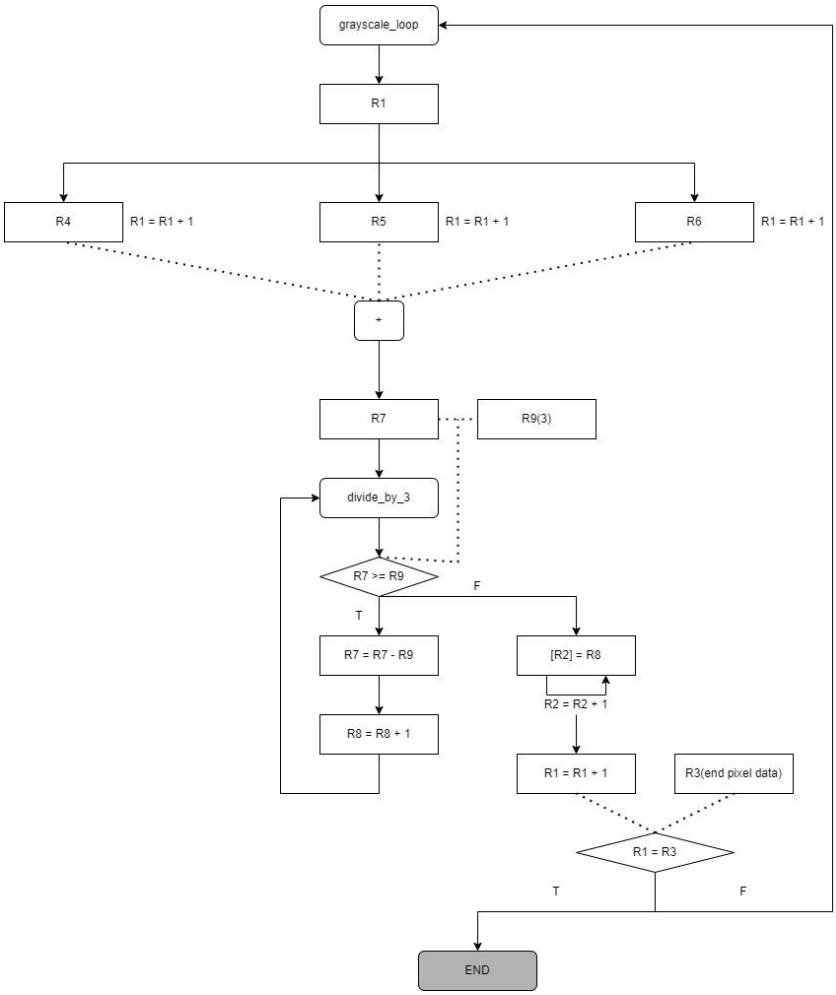
→ 본 프로젝트에서는 **(1)**에 해당하는 코드를 사용하여 프로젝트를 구현하였다.

2. 이미지 변환 함수의 동작 방식 및 Block diagram

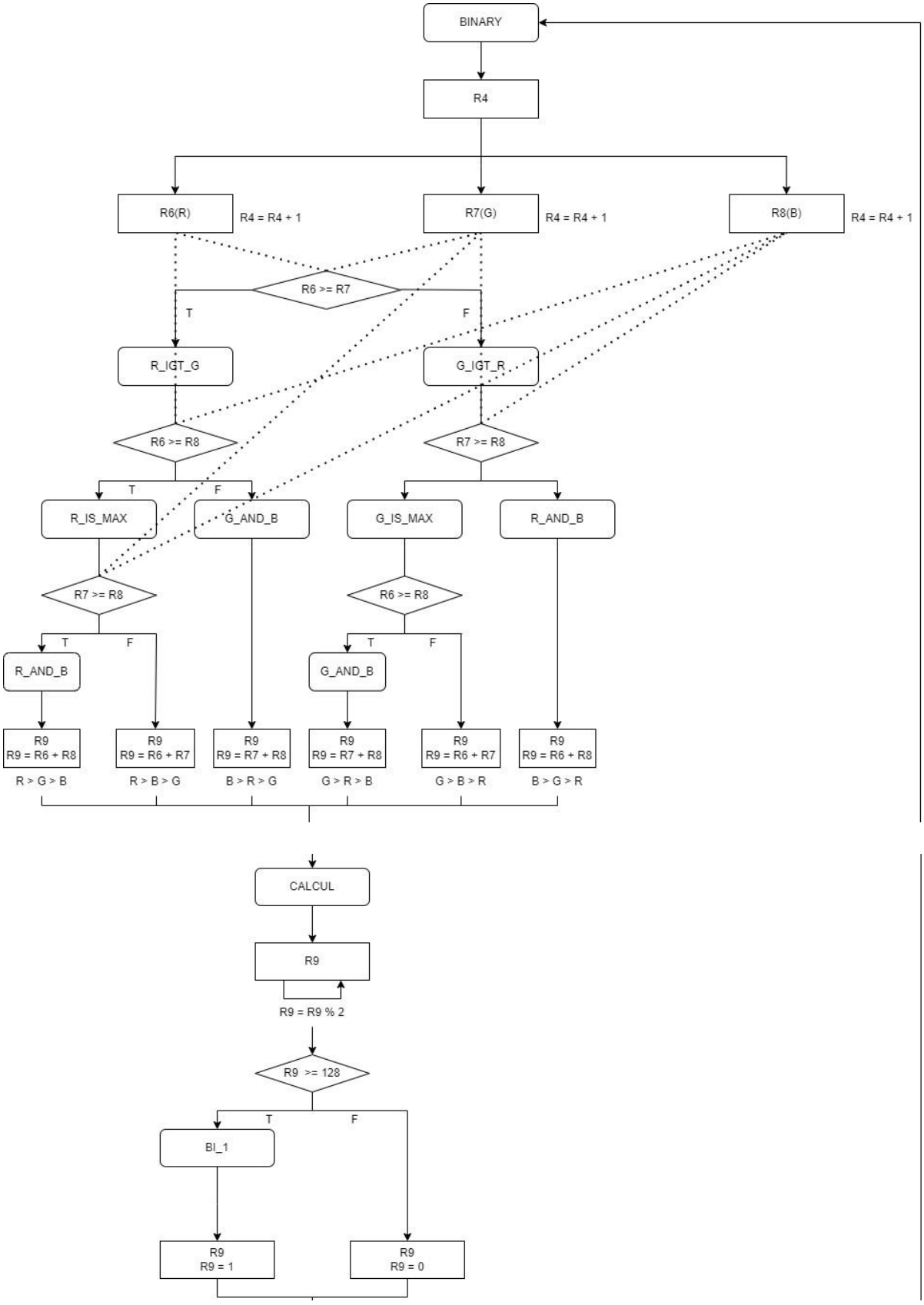
(1) Function 1 : 32-bit RGBA image → 8-bit RGB image

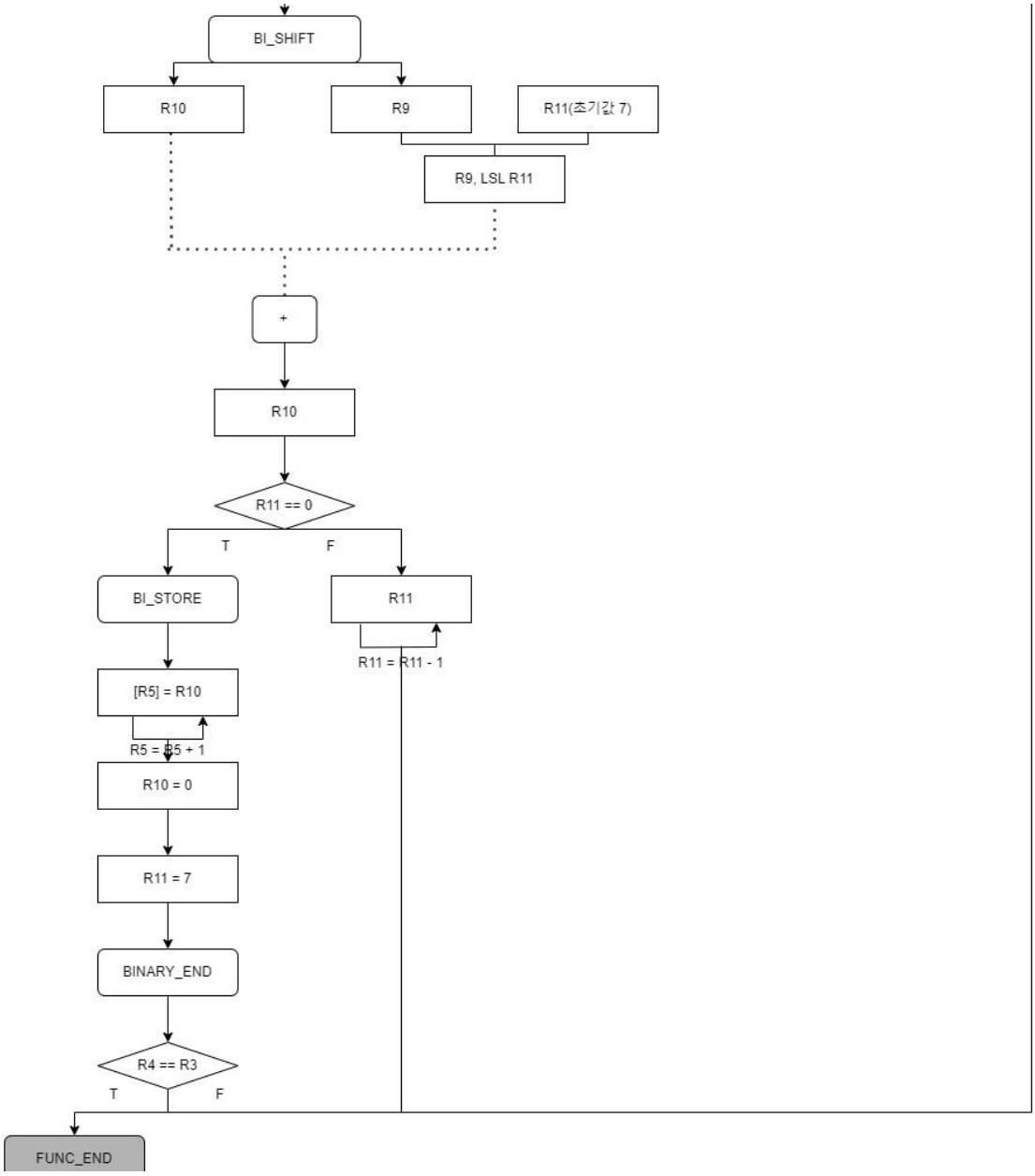


(2) Function 2 : 32-bit RGBA image → Grayscale image



(3) Function 3 : 32-bit RGBA image -> Binary image




BI_SHIFT
R10
R9
R11(초기값 7)
R9, LSL R11
+
R10
R11 == 0
T
F
BI_STORE
R11
R11 = R11 - 1
[R5] = R10
R5 = R5 + 1
R10 = 0
R11 = 7
BINARY_END
R4 == R3
T
F
FUNC_END

3. 3가지 이미지 변환 함수의 ARM Code

(1)-A. Function 1 : 32-bit RGBA image → 8-bit RGB image ARM Code

```
56          ; Here is starting point of Function 1, Convert to RGB
57
58          MOV R4, #0x30000000 ; From 0x30000000 to value of R3 (0x30096000)
59          MOV R5, #0x31000000 ; Memory address of "Storing RGB Byte"
60  RGB_END
61          CMP R4, R3 ; If R3 == R4 == 0x30096000, Converting is over
62          BEQ FUNC_END
63
64  RGB
65          LDRB R6, [R4], #1 ; Load R of RGBA
66          LDRB R7, [R4], #1 ; Load G of RGBA
67          LDRB R8, [R4], #1 ; Load B of RGBA
68          MOV R9, #0 ; We will store RGB here
69          ADD R4, R4, #1 ; Skip A of RGBA
70          AND R6, R6, #0xE0 ; Extract R[7:5]
71          AND R7, R7, #0xE0 ; Extract G[7:5]
72          LSR R7, R7, #3 ; Shift to make RGB
73          AND R8, R8, #0xC0 ; Extract B[7:6]
74          LSR R8, R8, #6 ; Shift to make RGB
75          ADD R9, R9, R6 ; R6 + R7 + R8 = RGB
76          ADD R9, R9, R7
77          ADD R9, R9, R8
78          STRB R9, [R5], #1 ; Store RGB Byte
79          B RGB_END ; Check it is over
80
81  FUNC_END
82          END
```

* 해당 Code 위에 있는 이미지를 Load하는 Code (본론 – 1)은 생략한다.

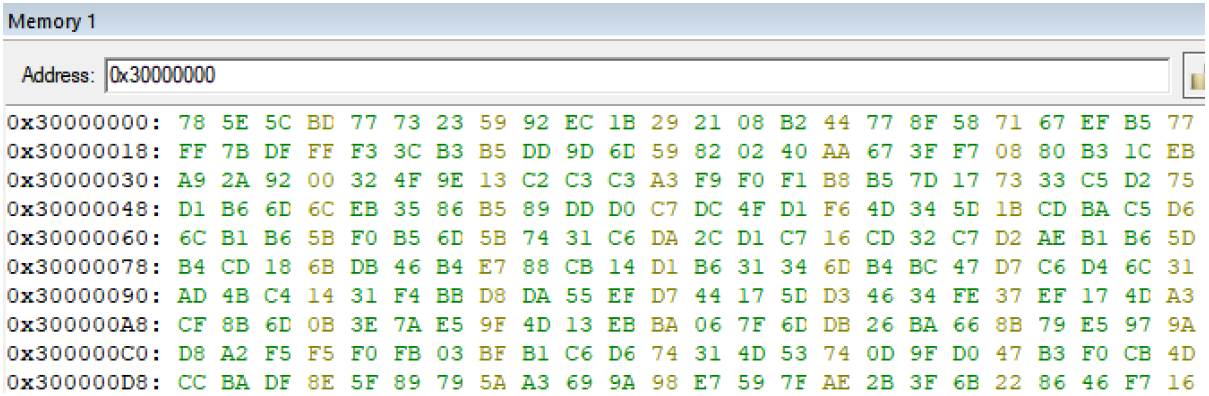
R4는 이미지에서 Header를 분리한 Pixel 데이터에 접근하기 위한 주소를 다루는 Register이고 R5는 RGB 8-bit로 변환한 Pixel 데이터를 저장하기 위한 주소를 다루는 Register이다. 'RGB_END' 레이블은 모든 Pixel 데이터 (960 * 160 * 4 Bytes)에 접근한 경우 함수 종료를 위한 구간이다. 'RGB' 레이블은 32-bit RGBA Pixel을 8-bit Pixel로 변환하는 구간이다.

MOV 명령어를 이용하여 R4 Register 에 이미지에서 Header 를 분리한 Pixel 데이터에 접근하기 위한 메모리 시작 주소 0x30000000 을 설정한다. R5 Register 에 8-bit RGB 데이터를 저장할 메모리의 시작 주소를 설정한다. 'RGB' 레이블은 RGBA 데이터에서 각 색상 요소를 Load 하는 곳이다. LDRB 명령어로 R4 가 가리키는 메모리 주소에서 1 byte (R, G, B 각각)씩 Load 하고, R4 주소를 1 씩 증가시킨다. MOV R9, #0 을 이용하여 R9 을 0 으로 초기화한다. ADD R4, R4, #1 은 Alpha (투명도) byte 를 건너뛰기 위하여 R4 를 한

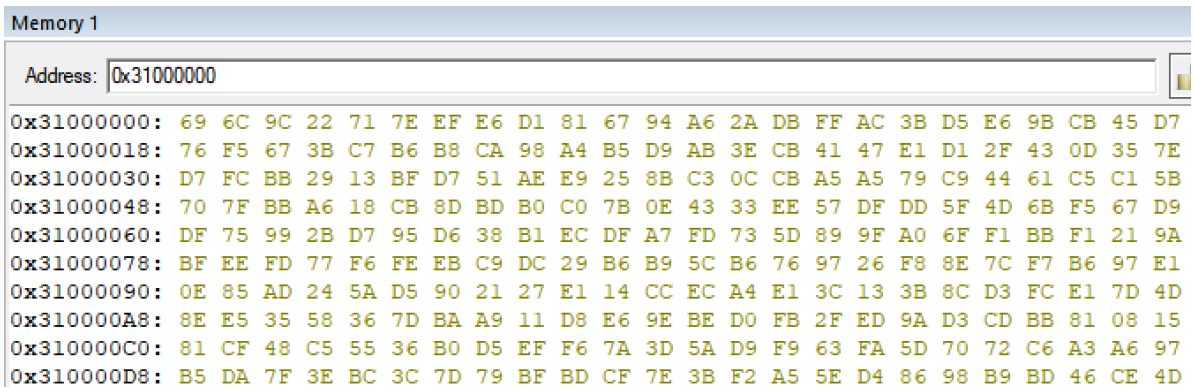
번 더 증가시키는 명령어이다. AND 명령어로 각 색상의 상위 비트를 추출한다. (R 과 G 는 상위 3 비트, B 는 상위 2 비트) LSR 명령어는 G 와 B 값을 적절한 비트 위치로 이동시켜 RGB 값으로 합하기 쉽게 한다. ADD 명령어로 최종 RGB 값을 계산한다. STRB 명령어로 최종 RGB 값을 R5 가 가리키는 주소에 저장하고 R5 주소를 1 증가시킨다. B 명령어로 'RGB_END' 레이블로 점프하여 변환 과정이 끝났는지 확인한다. R4 와 R3 가 똑같다는 것은 마지막 Pixel 데이터까지 모두 변환이 완료되었다는 것으로 'FUNC_END' 레이블로 이동한다. 이 레이블은 함수의 끝을 나타낸다. END 명령어로 함수를 종료한다.

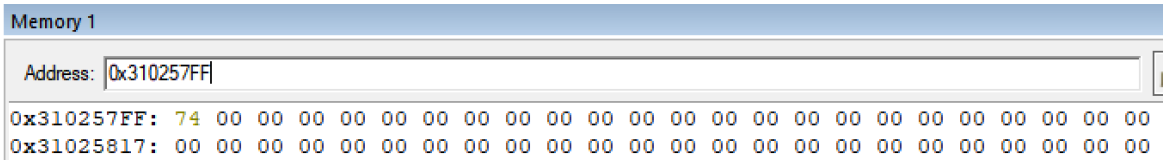
(1)-B. Memory map

다음 이미지는 기존 32-bit RGBA Pixel 이 저장된 곳의 Memory Map (0x30000000 ~ 0x30095FFF)이다. 초록색 글씨는 RGB 값으로 초록색은 한 번 이상 접근하였음을 의미한다. 노란색 글씨는 A값으로 노란색은 저장된 이후 접근하지 않았음을 의미한다.



다음 이미지는 새로운 8 – bit RGB Pixel 이 저장된 곳의 Memory Map (0x3100000 ~ 0x310257FF)이다.





(2)-A. Function 2 : 32-bit RGBA image → Grayscale image ARM Code

```
53        MOV R1, #0x30000000       ; Start address of pixel data
54        MOV R2, #0x32000000       ; Destination address for grayscale data
55
56  grayscale_loop
57        LDRB  R4, [R1], #1        ; Load Red value
58        LDRB  R5, [R1], #1        ; Load Green value
59        LDRB  R6, [R1], #1        ; Load Blue value
60        ADD   R7, R4, R5          ; R7 = Red + Green
61        ADD   R7, R7, R6          ; R7 = R7 + Blue
62
63        ; Divide by 3 using subtraction loop
64        MOV   R8, #0              ; Initialize result of division
65        MOV   R9, #3              ; Divisor = 3
66
67  divide_by_3
68        CMP   R7, R9              ; Check if R7 >= R9
69        SUBCS R7, R7, R9          ; R7 -= R9 (subtraction if carry set)
70        ADDCS R8, R8, #1          ; R8 += 1 (increment result if carry set)
71        BCS   divide_by_3         ; Loop back if carry set
72
73        STRB  R8, [R2], #1        ; Store the grayscale value
74        ADD   R1, R1, #1          ; Skip Alpha channel
75
76        CMP   R1, R3              ; Check if end of pixel data
77        BLT   grayscale_loop      ; Loop if not end
78
79        END
```

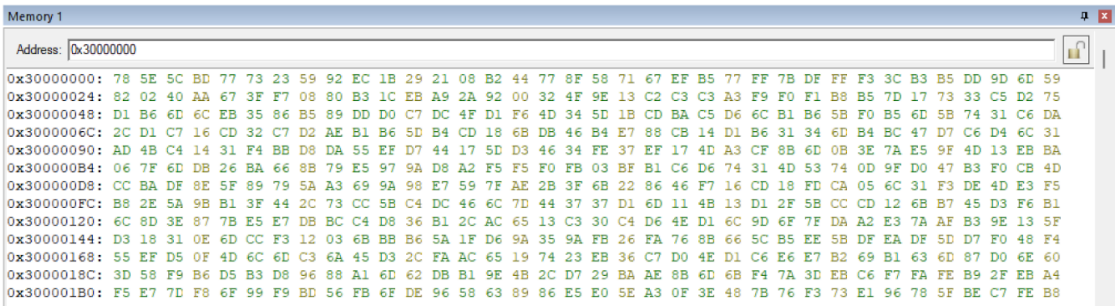
R1는 Image에서 Header를 분리한 Pixel 데이터에 접근하기 위한 주소를 다루는 Register이고 R2는 Grayscale로 변환한 Pixel 데이터를 저장하기 위한 주소를 다루는 Register이다. 'grayscale_loop' 레이블은 RGB 데이터를 Load 하여 변환과정을 준비하는 구간이다. 'divide_by_3' 레이블은 Grayscale value로 직접 변환을 하는 구간이다.

R1 Register에 메모리 시작 주소 0x30000000을 Load 한다. 이 주소는 Image에서 Header를 분리한 Pixel 데이터가 저장된 시작 위치이다. R2 Register에는 Grayscale 데이터를 저장할 목적지 주소인 0x32000000을 Load 한다. 'grayscale_loop' 레이블은 Grayscale 변환을 수행하는 Loop의 시작점이다. LDRB 명령은 R1 Register가 가리키는 주소에서 R, G, B 값을 각각 Load 하고, Load 후 R1 주소를 1씩 증가시킨다. ADD 명령은 R, G, B 값을 합하여 R7에 저장한다. 3으로 나누는 과정을 위해 R8을 결과 저장용으로 0으로 초기화하고, R9에 나누는 수인 3을 Load 한다. 'divide_by_3' 레이블은 나눗셈을 수행하는 Loop이다. CMP 명령으로 R7과 R9을 비교한다. SUBCS는 조건부 명령으로, R7이 R9

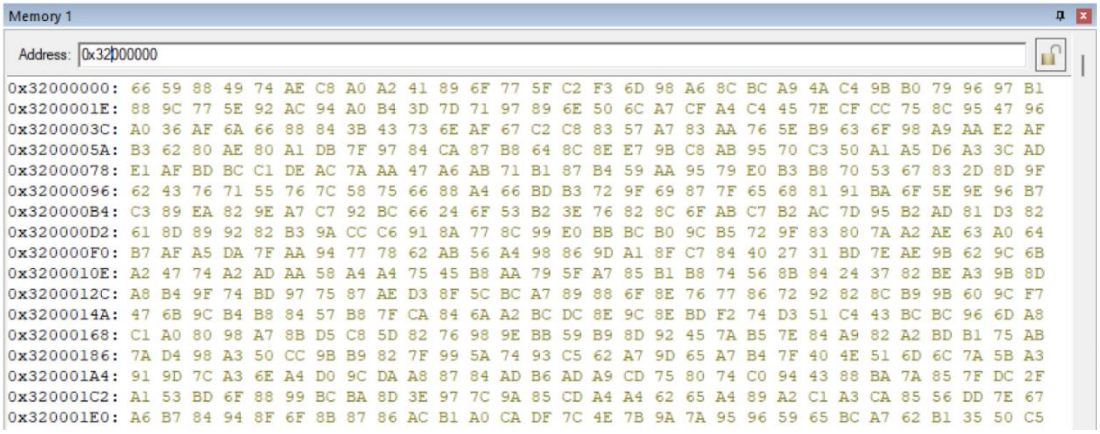
이상일 때만 R7에서 R9를 뺀다. ADDCS는 R7이 R9 이상일 경우 R8을 1 증가시킨다. R8은 나눗셈의 최종 몫을 저장해야 하는 Register 이기 때문에 R8을 나눗셈이 수행될 때마다 1씩 증가시키는 것이다. BCS는 나눗셈을 계속할 조건이 충족일 때 'divide_by_3'로 점프한다. STRB 명령으로 계산된 Grayscale 값 R8을 R2가 가리키는 주소에 저장하고 R2 주소를 1 증가시킨다. ADD 명령으로 알파 채널을 건너뛰기 위해 R1 주소를 1 증가시킨다. CMP 명령으로 R1과 R3을 비교하여 전체 Pixel 데이터 처리가 완료되었는지 확인한다. BLT는 R1이 R3보다 작으면 'grayscale_loop'로 점프하여 계속 처리한다. END는 프로그램의 끝을 나타낸다. 이 명령어를 통해 함수를 종료한다.

(2)-B. Memory map

다음은 기존 32-bit RGBA Pixel 이 저장된 곳의 Memory Map (0x30000000 ~ 0x30095FFF)이다. 초록색 글씨는 RGB 값으로 한 번 이상 접근하였음을 의미한다. 노란색 글씨는 A 값으로 접근을 하지 않았음을 의미한다.



다음은 새로운 Grayscale Pixel 이 저장된 곳의 Memory Map (0x3200000 ~)이다.



(3)-A. Function 3 : 32-bit RGBA image -> Binary image ARM Code

```
        ; Here is starting point of Function 3, Converting to Binary

        MOV R4, #0x30000000 ; From 0x30000000 to value of R3 (0x30096000)
        MOV R5, #0x31000000 ; Memory address of "Storing Binary Byte"
        MOV R10, #0 ; Register storing binary bits (8 Binary Pixel = 1 Byte)
        MOV R11, #7 ; Value for shift to combine binary bits

BINARY_END
        CMP R4, R3 ; If not R4 != R3 == 0x30096000
        BNE BINARY ; Need more operation, Go to BINARY
        B FUNC_END ; Else if R4 == R3 == 0x30096000, Function finished! Go to FUNC_END

BINARY
        LDRB R6, [R4], #1 ; Load R of RGBA
        LDRB R7, [R4], #1 ; Load G of RGBA
        LDRB R8, [R4], #1 ; Load B of RGBA
        MOV R9, #0 ; We will store ( max(R, G, B) + min(R, G, B) ) here
        ADD R4, R4, #1 ; Skip A of RGBA
        CMP R6, R7 ; Compare R with B
        BGE R_IGT_G ; R >= G
        B G_IGT_R ; R < G

R_IGT_G ; R >= G
        CMP R6, R8
        BGE R_IS_MAX ; R > G, B
        B G_AND_B ; B > R > G

G_IGT_R ; G > R
        CMP R7, R8
        BGE G_IS_MAX ; G > R, B
        B R_AND_B ; B > G > R

R_IS_MAX ; R > G, B
        CMP R7, R8
        BGE R_AND_B ; R > G > B
        ADD R9, R9, R6 ; R > B > G
        ADD R9, R9, R7
        B CALCUL

G_IS_MAX ; G > R, B
        CMP R6, R8
        BGE G_AND_B ; G > R > B
```

```
        ADD R9, R9, R6 ; G > B > R
        ADD R9, R9, R7
        B CALCUL

R_AND_B
        ADD R9, R9, R6
        ADD R9, R9, R8
        B CALCUL

G_AND_B
        ADD R9, R9, R7
        ADD R9, R9, R8
        B CALCUL

CALCUL
        LSR R9, R9, #1
        CMP R9, #128
        BGE BI_1 ; If R9 > 127, Current binary bit is 1
        MOV R9, #0 ; Else if R9 <= 127, Current binary bit is 0
        B BI_SHIFT

BI_1
        MOV R9, #1 ; R9 > 127, Current binary bit is 1

BI_SHIFT
        ADD R10, R10, R9, LSL R11
        CMP R11, #0 ; If R11 == 0, R11 = 7
        BEQ BI_STORE
        SUB R11, R11, #1 ; R11 -= 1
        B BINARY

BI_STORE
        STRB R10, [R5], #1
        MOV R10, #0
        MOV R11, #7
        B BINARY_END

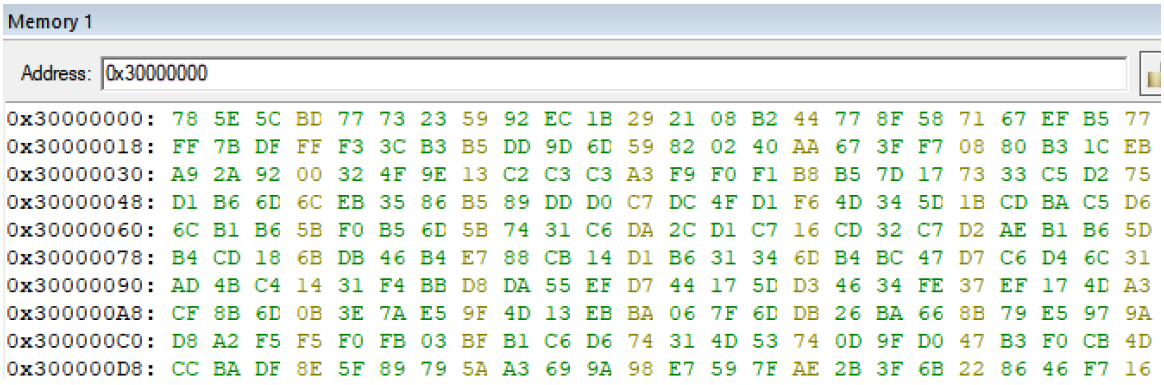
FUNC_END
        END
```

R4는 Image에서 Header를 분리한 Pixel 데이터에 접근하기 위한 주소를 가지고 있는 Register이다. R5는 8개의 1 Bit Binary Pixel을 1 Byte로 저장하기 위한 주소를 가지고 있는 Register이다. 'BINARY_END' 레이블은 모든 Pixel 데이터(960 * 160 * 4 Bytes)에 접근한 경우 함수 종료를 위한 구간이다. 'BINARY' 레이블은 32-bit RGBA Pixel을 Binary Pixel로 변환하는 구간이다. 'CALCUL' 레이블은 127보다 크면 1, 작거나 같으면 0으로 변환하는 구간이다.

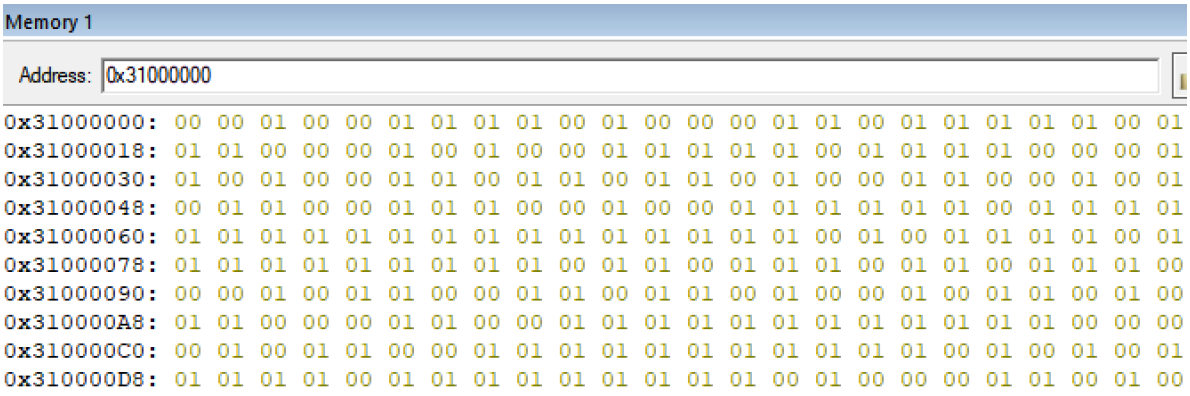
R4 Register 에 메모리 시작 주소 0x30000000 을 Load 한다. 이 주소는 Image 에서 Header 를 분리한 Pixel 데이터가 저장된 시작 위치이다. R5 Register 에는 8 개의 1-bit Binary Pixel 을 1-Byte 로 저장하기 위한 주소를 Load 한다. R10 Register 는 8 개의 Binary pixel 을 저장하는 Register 이다. R11 은 8 비트 바이너리 값을 구성하기 위해 사용될 Shift 값으로 초기에 7 로 설정된다. 'BINARY_END' 레이블은 바이너리 변환 과정의 끝을 나타낸다. CMP 와 BNE 는 R4 와 R3 의 값이 다르면 계속해서 'BINARY' 레이블로 점프하여 변환을 계속 진행하도록 한다. R4 와 R3 가 같다면 모든 데이터의 처리가 완료된 것으로, 'FUNC_END'로 점프하여 함수를 종료한다. 'BINARY' 레이블은 RGBA 데이터에서 R, G, B 값을 각각 Load 한다. R9 는 연산 결과를 임시 저장할 Register 로 초기화된다. ADD 를 이용해 R4 를 증가시켜 Alpha (투명도) byte 를 건너뛴다. R 과 G 를 비교하여 R 이 크거나 같으면 'R_IGT_G'로, 그렇지 않으면 'G_IGT_R' 로 분기한다. 'R_IGT_G'에서 R 과 B 를 비교하여 R 이 B 보다 크거나 같으면, 'R_IS_MAX'로, 그렇지 않으면 'G_AND_B'로 분기한다. 이하의 'G_IGT_R', 'R_IS_MAX', 'R_AND_B', 'G_AND_B' 레이블은 비슷한 방식으로 각 색상 값의 최대값과 최소값을 비교하여 계산하는 로직을 포함하고 있다. 'CALCUL' 레이블은 R9 의 값을 오른쪽으로 1-bit Shift (2 로 나누는 효과)한 뒤, 이를 128 과 비교하여 127 이상이면 1 로, 아니면 0 으로 설정한다. 'BI_SHIFT'는 계산된 바이너리 비트를 R10 에 Shift 하여 저장하고, 이를 8 비트로 완성때까지 반복한다. 완성된 8 비트 바이너리 값을 R5 가 가리키는 주소에 저장하고, 다음 바이트를 처리하기 위해 R10 과 R11 을 초기화 한 뒤 'BINARY_END'로 분기한다.

(3)-B. Memory map

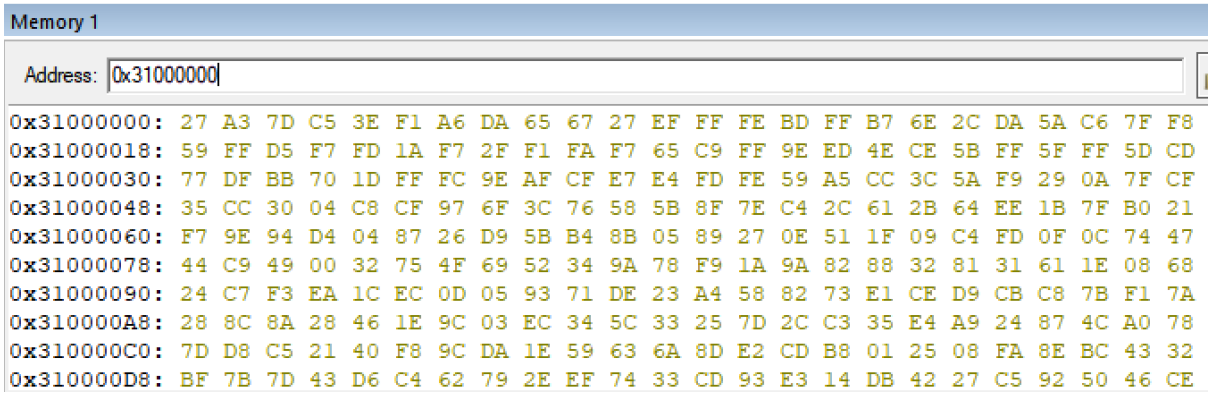
다음은 기존 32-bit RGBA Pixel 이 저장된 곳의 Memory Map (0x30000000 ~ 0x30095FFF)이다. 초록색 글씨는 RGB 값으로 한 번 이상 접근하였음을 의미한다. 노란색 글씨는 A 값으로 접근을 하지 않았음을 의미한다.

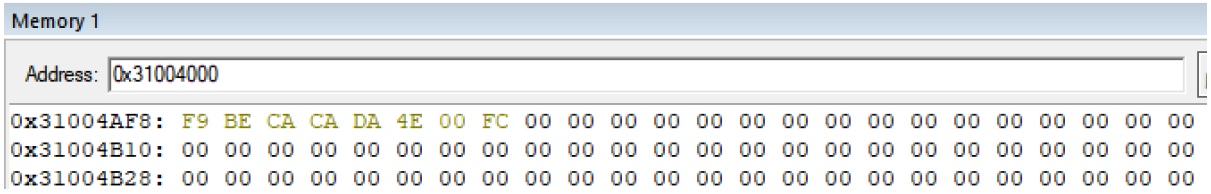


다음은 32-bit RGBA Pixel 이 Binary Pixel 로 변환된 모습이다. **실제 저장된 모습은 아니다.** 해당 Memory Map 은 가독성을 위한 예시로 하나의 Binary Pixel 이 1 Byte 의 크기를 가지고 있다. 실제 Binary Pixel 은 1 Bit 이므로 8 개의 Binary Pixel 이 한 주소에 저장된다.



다음은 실제 32-bit RGBA Pixel이 1-Bit Binary Pixel로 변환되어 저장된 Memory Map (0x31000000 ~ 0x31004AFF)이다. 940 * 160 * 4 (기존 RGBA) Bytes가 32배 감소하였다.





4. Memory Relocation과 기능 구현을 위한 ARM Code

(1)-A. Function 4 : Memory Relocation

```
1          AREA RESET, DATA, READONLY
2          AREA Image, CODE, READONLY
3
4          ENTRY
5          ; Initialization
6
7          MOV R1, #0x40000000 ; Starting Address of Input (Image)
8          MOV R2, #0x20000000 ; Memory Address of "Storing HEADER"
9          MOV R3, #0x30000000 ; Memory Address of "Storing R"
10         MOV R4, #0x31000000 ; Memory Address of "Storing G"
11         MOV R5, #0x32000000 ; Memory Address of "Storing B"
12         ; Clear register to use for loading and storing
13         MOV R6, #0
14         MOV R7, #0
15         MOV R11, #0
16         B HEADER
```

Memory Relocation 하기 전에 기본 정보를 세팅을 한다. R1에 이미지를 메모리에 로드하기 위해 이미지 시작 주소를 설정한다. R2에 이미지에서 헤더를 분리하여 저장할 메모리 주소를 설정한다. R3에 Register에 R 값을 저장할 메모리 주소를 설정한다. 동일하게 R4에는 G, R5에는 B 값을 저장할 메모리 주소를 설정한다. R6, R7, R11 값을 0으로 초기화한다. 'HEADER'로 분기한다.

```
43 HEADER ; Store bytes of HEADER from image
44         LDRB R6, [R1]
45         STRB R11, [R1], #1
46         MOV R7, R6
47         STRB R6, [R2], #1
48         CMP R7, #0x54 ; Check that current byte is 'T'
49         BNE HEADER ; If it is not 'T', keep loading
50         B HEADER_END ; If it is 'T', check if it is 'T' of "IDAT"
```

HEADER에서는 이미지의 헤더를 저장한다. R1에서 값을 Load하여 R6에 저장한다. 0을 R1 값을 주소 값으로 하는 메모리에 저장하고 R1을 증가시킨다. R6의 값을 R7으로 이동시킨다. R6의 값을 R2에 저장하고 R2를 증가시킨다. R7의 값을 ASCII 값 'T'와 비교한다. T가 아니면 헤더 로딩을 계속하고 T이면 'HEADER_END'로 분기하여 'IDAT'를 판별한다. 'IDAT'는 Pixel의 시작을 나타내는 값으로 해당 값 이후부터 Pixel(RGBA) 값이므로 분리가

필요하다고 판단하여 전과 후의 값을 다른 곳에 저장한다.

```
18  HEADER_END ; Check "IDAT"
19          ; We will check 3 bytes before 'T'
20          SUB R2, R2, #2
21          MOV R10, #0
22          LDRB R6, [R2], #-1 ; Check if it is 'A'
23          MOV R9, R6
24          CMP R9, #0x41
25          ADDNE R10, R10, #1 ; If it is not 'A', mark at R10
26          LDRB R7, [R2], #-1 ; Check if it is 'D'
27          MOV R9, R7
28          CMP R9, #0x44
29          ADDNE R10, R10, #1 ; If it is not 'D', mark at R10
30          LDRB R8, [R2], #-1 ; Check if it is 'I'
31          MOV R9, R8
32          CMP R9, #0x49
33          ADDNE R10, R10, #1 ; If it is not 'I', mark at R10
34          ; Restore to original state
35          STRB R6, [R2, #1]!
```

```
36          STRB R7, [R2, #1]!
37          STRB R8, [R2, #1]!
38          ADD R2, R2, #2
39          CMP R10, #0 ; If R10 != 0, it is not "IDAT"
40          MOV R7, #0
41          BEQ PIXEL ; If it is "IDAT", terminate loading HEADER and start loading PIXEL
```

HEADER_END에서는 "IDAT"을 확인한다. 현재 위치에서 2바이트 이전을 가리키도록 R2를 조정한다. R10의 값을 0으로 초기화한다. R2에서 바이트를 Load하여 R6에 저장하고 R2를 감소시켜 'A'인지 확인한다. R6의 값을 R9로 이동시킨다. R9값을 ASCII 값 'A'와 비교한다. A가 아니면 R10을 1 증가시킨다. R2에서 값을 Load하여 R7에 저장하고 R2를 감소시켜 'D'인지 확인한다. D가 아니면 R10을 1 증가시킨다. R2에서 값을 Load하여 R8에 저장하고 R2를 감소시켜 'I'인지 확인한다. R8의 값을 R9로 이동시킨다. R9 값을 ASCII 값 'I'와 비교한다. I가 아니면 R10을 1 증가시킨다. R6의 값을 R2에 저장하고 R2를 1 증가시킨다. R7과 R8도 동일하게 수행한다. R2를 원래 위치로 조정한다. R10의 값을 0과 비교한다. R7값을 0으로 초기화한다. 모든 값이 일치하면 'PIXEL' 레이블로 분기한다.

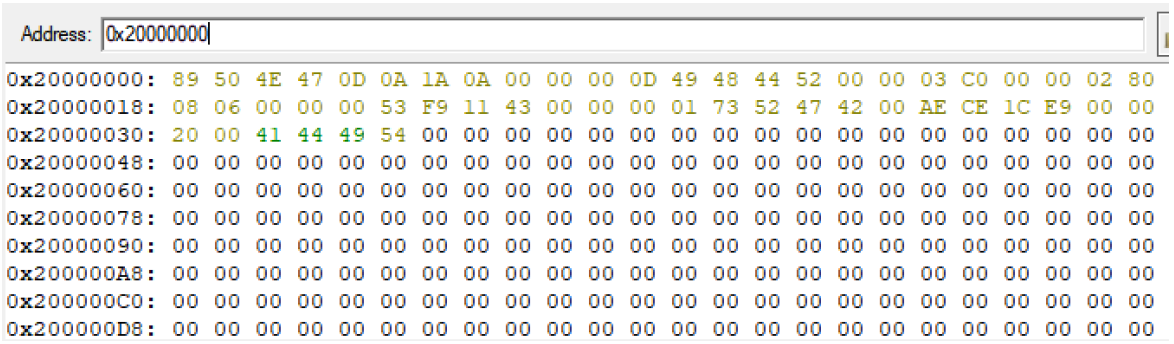
```
52  PIXEL ; Store bytes of PIXEL from image
53          LDRB R6, [R1] ; Store R of RGBA
54          STRB R11, [R1], #1
55          STRB R6, [R3], #1
56          LDRB R6, [R1] ; Store G of RGBA
57          STRB R11, [R1], #1
58          STRB R6, [R4], #1
59          LDRB R6, [R1] ; Store B of RGBA
60          STRB R11, [R1], #1
61          STRB R6, [R5], #1
62          ADD R1, R1, #1 ; Skip A of RGBA
63          ADD R7, #4
64          CMP R7, #614400 ; 614400 = 960 * 160 * 4 (Bytes)
65          BNE PIXEL
```

'PIXEL' 레이블에서는 이미지에서 픽셀 바이트를 저장한다. RGBA의 빨간색 값(R)을 저장하기 위해 R1에서 값을 Load하여 R6에 저장한다. 0을 R1에 저장하고 R1을 1만큼 증가

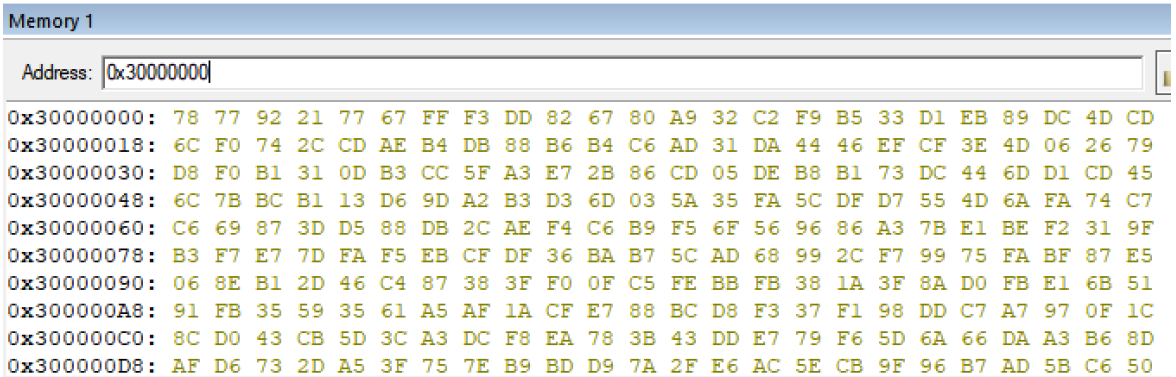
시킨다. 빨간색 값을 R6에서 R3에 저장하고 R3을 증가시킨다. 초록색과 파란색도 각각 R4, R5에 저장하여 동일하게 연산한다. R1을 증가시켜 RGBA의 A 값을 건너뛴다. R7을 4만큼 증가시킨다. R7과 61,440 바이트, 즉 픽셀의 총 바이트 수를 비교하여 같지 않으면 픽셀 로드를 계속한다. 같다면 프로그램을 종료한다.

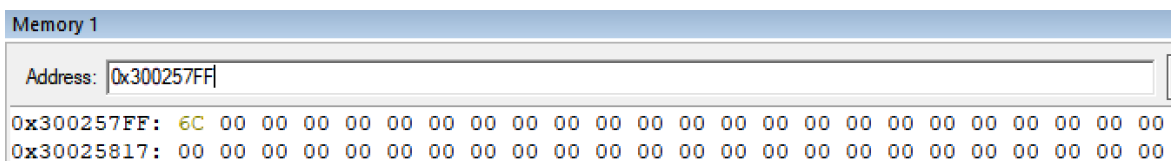
(1)-B. Memory map

다음은 Header 바이트가 저장된 곳의 Memory Map (0x20000000~0x20000035)이다. 초록색 글씨는 각각 'I', 'D', 'A' 값으로 "IDAT" 판단을 위해 접근했음을 의미한다.

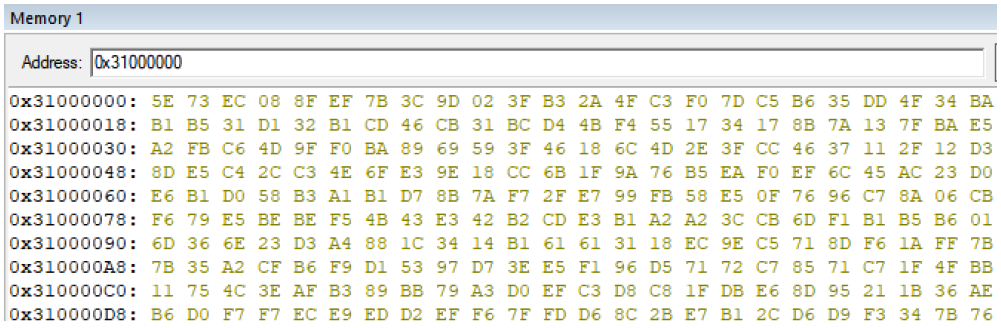


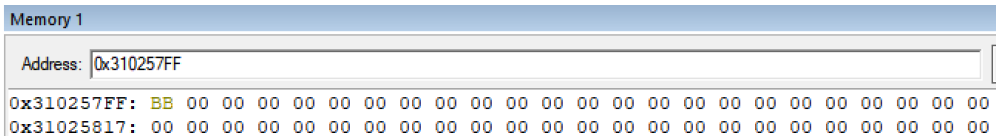
다음은 8비트 'R' 바이트들이 저장된 곳의 Memory map (0x30000000~0x300247FF)이다. 960 x 160 바이트만큼의 'R'이 저장된다.



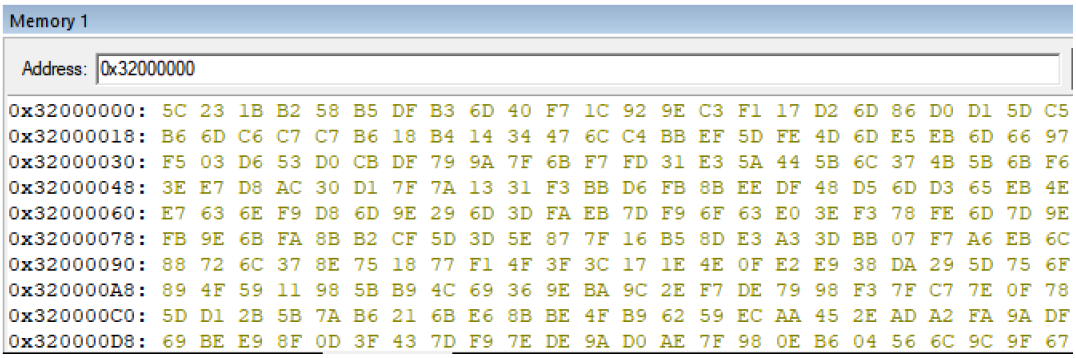


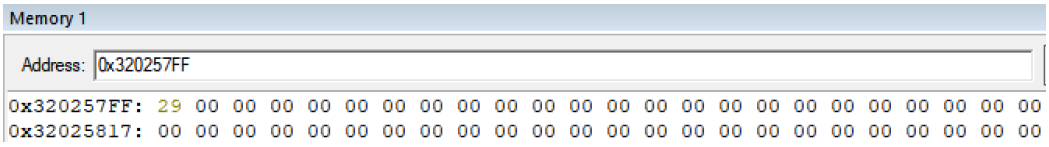
다음은 8비트 'G' 바이트들이 저장된 곳의 Memory map (0x3100000~0x310257FF)이다. 960 x 160 바이트만큼의 'G'이 저장된다.



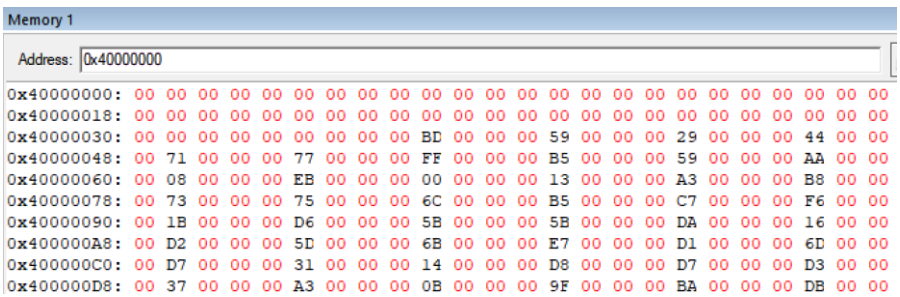


다음은 8비트 'B' 바이트들이 저장된 곳의 Memory map (0x32000000~0x320257FF)이다. 960 x 160 바이트만큼의 'B'이 저장된다.





다음은 이미지 HEX 파일이 있던 주소의 Memory map (0x40000000~)이다. Header와 R, G, B는 추출되고 A만 추출되지 않은 모습을 확인할 수 있다.



(1)-C. A (Alpha) 제외 특이점

실제 과정에서 A를 제외한 R, G, B로만 데이터를 관리할 경우 메모리 주소 값을 4 byte, word 단위로 관리하는 ARM에서 데이터를 관리하기 까다로운 점이 발생한다. R, G, B 값만이 연속적으로 존재할 경우 3 byte 단위로 데이터를 관리해야 하는데 주소 값을 4 byte, word 단위로 관리하는 ARM에서는 데이터를 저장하는 것 자체도 어렵거나 불가능하고, 실제 함수를 구현하게 되더라도 명령어나 주소 값 설정 및 관리 측면에서 큰 어려움이 있다. 따라서, A를 제외하고 R, G, B 정보만 연속적인 메모리 공간에 저장하는 것은 Word-aligned 측면에서의 문제가 있다.

Memory Relocation 과정에서 A를 제외하면 이미지에는 이러한 특징들이 나타나기도 한다. 첫째, 색상 왜곡이 없다. A를 제외하는 것은 색상 정보 (R, G, B)에 영향을 주지 않는다. 따라서 색상의 왜곡이나 손실은 발생하지 않는다. 이미지의 색상은 동일하게 유지된다. 둘째, 호환성이 증가한다. 일부 애플리케이션이나 디스플레이 장치는 24비트 RGB 형식만 지원한다. 따라서 RGBA를 RGB로 변환하면 더 많은 환경에서 호환성을 확보할 수 있다. 셋째, 파일 크기가 감소한다. 32비트 RGBA 이미지에서 A를 제거하면 각 픽셀이 4바이트에서 3바이트로 줄어들어 전체 파일 크기가 약 25% 감소한다. 예를 들어, 원본 RGBA 이미지가 1MB였다면, RGB 이미지로 변환하면 약 0.75MB가 된다. 파일 크기가 감소하지만 RGB만 존재하면 3byte이기 때문에 컴파일러가 최적화를 하지 못한다. 또한 투명도 정보 손실이 발생한다. RGBA에서 A는 픽셀의 투명도(알파 값)를 나타내는데 A를 제거하면 이 투명도 정보가 완전히 사라지게 된다. 이는 이미지가 더 이상 반투명이나 투명한 부분을 표현할 수 없게 되며 모든 픽셀이 불투명하게 된다.

(2)-A. Function 4-1 : Function 4가 적용된 8-bit RGB ARM Code

```
68          ; Here is starting point of Function 1, Convert to RGB
69
70          MOV R7, #0x30000000 ; Memory address of R
71          MOV R8, #0x31000000 ; Memory address of G
72          MOV R9, #0x32000000 ; Memory address of B
73          MOV R11, #0x25000000 ; Memory address of 8-bit RGB
```

빨간색(R)을 저장할 메모리 주소를 R7으로 설정한다. 초록색(G), 파란색(B)은 각각 R8, R9에 설정한다. 8비트 RGB를 저장할 메모리 주소를 R11에 설정한다.

```
74  RGB_END
75          CMP R7, R3 ; If R7 == R3 , Converting is over
76          BNE RGB
77          CMP R8, R4 ; If R8 == R4 , Converting is over
78          BNE RGB
79          CMP R9, R5 ; If R9 == R5 , Converting is over
80          BEQ FUNC_END
```

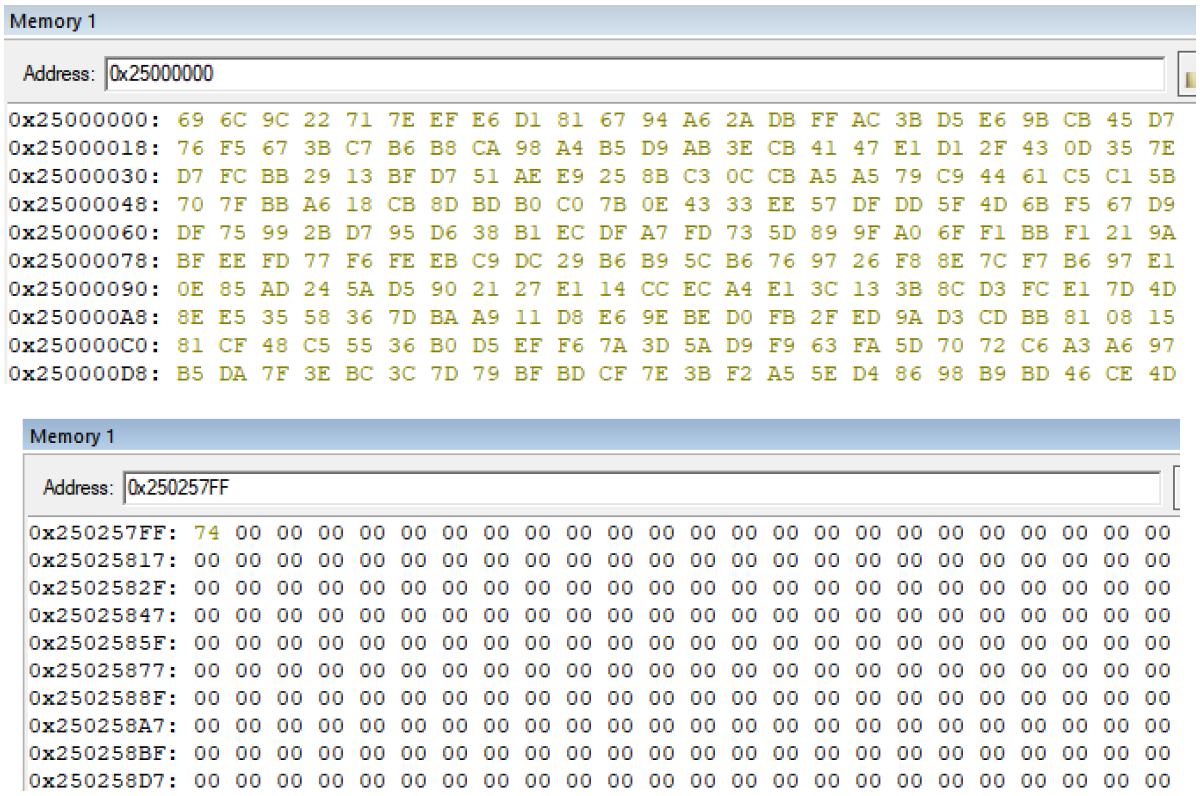
'RGB_END'는 모든 픽셀 데이터에 접근했을 때 함수 종료를 판단하기 위한 레이블이다. R7과 R3을 비교하여 값이 다르면 RGB 섹션으로 분기한다. R8과 R4를 비교하여 값이 다르면 RGB 섹션으로 분기한다. R9와 R5를 비교하여 같으면 'FUNC_END'로 분기한다.

```
82  RGB
83          MOV R10, #0 ; We will store RGB here
84          LDRB R6, [R7], #1 ; Load R of RGBA
85          AND R6, R6, #0xE0 ; Extract R[7:5]
86          ADD R10, R10, R6
87
88          LDRB R6, [R8], #1 ; Load G of RGBA
89          AND R6, R6, #0xE0 ; Extract G[7:5]
90          LSR R6, R6, #3 ; Shift to make RGB
91          ADD R10, R10, R6
92
93          LDRB R6, [R9], #1 ; Load B of RGBA
94          AND R6, R6, #0xC0 ; Extract B[7:6]
95          LSR R6, R6, #6 ; Shift to make RGB
96          ADD R10, R10, R6
97
98          STRB R10, [R11], #1 ; Store RGB Byte
99          B RGB_END ; Check it is over
100
101 FUNC_END
102         END
```

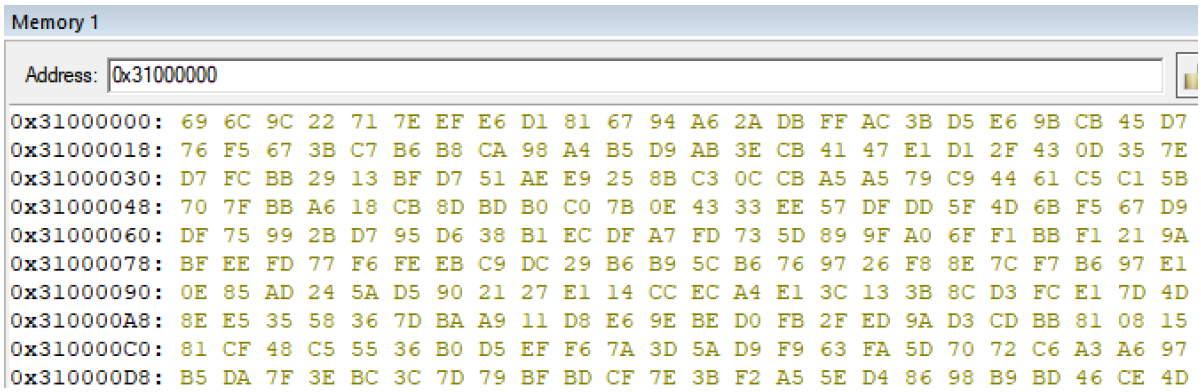
RGB는 8bit RGB로 변환하여 저장하는 구간이다. RGB 값을 저장할 R10을 0으로 초기화한다. R7에서 값을 Load하여 R6에 저장하고 R7을 1 증가시킨다. R[7:5] 비트(상위 3비트)를 추출한다. 추출한 R비트를 R10에 더한다. 같은 방법으로 초록색 파란색 비트를 추출하고 R10에 더할 때 각각 3비트, 6비트씩 이동시켜 R10에 더한다. R10의 RGB 값을 R11 메모리 주소에 저장하고 R11을 1증가시킨다. 변환 종료를 확인하기 위해 'RGB_END'로 분기한다. 프로그램을 종료한다.

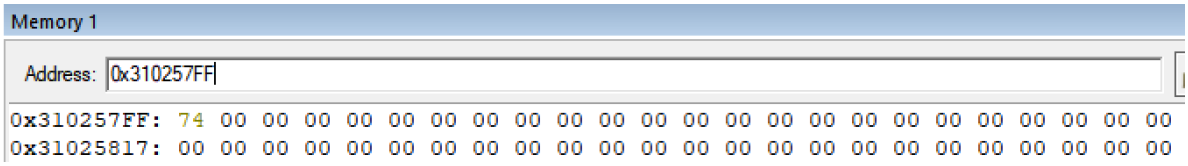
(2)-B. Memory map

8-bit RGB 변화를 수행하면 다음과 같은 결과가 나오게 된다.



이는 앞서 구현한 Function 1의 결과와 동일한 것을 볼 수 있다.





(3)-A. Function 4-2 : Function 4가 적용된 Grayscale ARM Code

```
69          MOV R1, #0x30000000 ; Memory address of R
70          MOV R2, #0x31000000 ; Memory address of G
71          MOV R3, #0x32000000 ; Memory address of B
72          MOV R4, #0x25000000 ; Memory address of 8-bit RGB
73          MOV R11, #0x31000000
74          MOV R5, #0
```

R, G, B 그리고 8비트 값을 저장할 메모리 주소를 각각 R1, R2, R3, R4로 설정한다. 루프의 종료주소, 즉 G값의 메모리 주소 끝을 의미하는 주소를 설정한다. R+G+B 값을 저장을 위한 Register를 초기화한다.

```
75  grayscale_loop
76          LDRB  R6, [R1], #1          ; Load Red value
77          LDRB  R7, [R2], #1          ; Load Green value
78          LDRB  R8, [R3], #1          ; Load Blue value
79          ADD   R5, R6, R7            ; R7 = Red + Green
80          ADD   R5, R5, R8            ; R7 = R7 + Blue
81
82          ; Divide by 3 using subtraction loop
83          MOV   R9, #0                ; Initialize result of division
84          MOV   R10, #3                ; Divisor = 3
```

'grayscale_loop'를 시작한다. R1 값(R)을 R6에 로드하고 R1을 1 증가시킨다. R2 값(G)을 R7에 로드하고 R2값을 1 증가시킨다. R3 값(B)을 R8에 로드하고 1 증가시킨다. R5에 R6(R)과 R7(G)의 합을 저장한다. R5에 현재 값과 R8(B)을 더하여 R+G+B의 합을 저장한다. 나눗셈 결과 저장을 위해 R9에 0, R10에 나눌 값인 3을 저장한다.

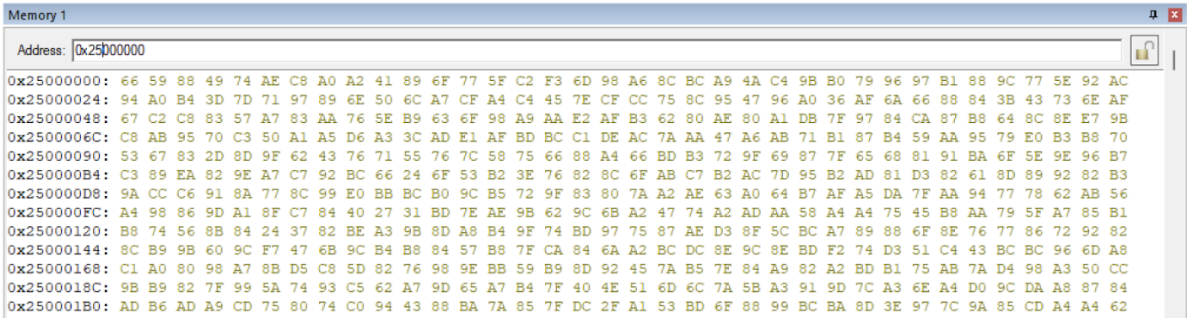
```
85
86  divide_by_3
87          CMP   R5, R10               ; Check if R7 >= R9
88          SUBCS R5, R5, R10           ; R7 -= R9 (subtraction if carry set)
89          ADDCS R9, R9, #1           ; R8 += 1 (increment result if carry set)
90          BCS   divide_by_3          ; Loop back if carry set
91
92          STRB  R9, [R4], #1         ; Store the grayscale value
93          ;ADD   R1, R1, #1           ; Skip Alpha channel
94
95          CMP   R1, R11               ; Check if end of pixel data
96          BLT   grayscale_loop       ; Loop if not end
97
98          ; Infinite loop
99          B     .
100
101         END
```

'divided_by_3' 레이블에서는 R5(R+G+B)가 R10(3)보다 크거나 같은 지 비교한다. 만약 R5>=R10이면 R5에서 3을 빼고 R9(나눗셈 결과)에 1을 더한 후 'divide_by_3'으로 다시 분기한다. R9를 R4 메모리 주소에 저장하고 R4를 1 증가시킨다. 알파 값은 건너뛴다. R1

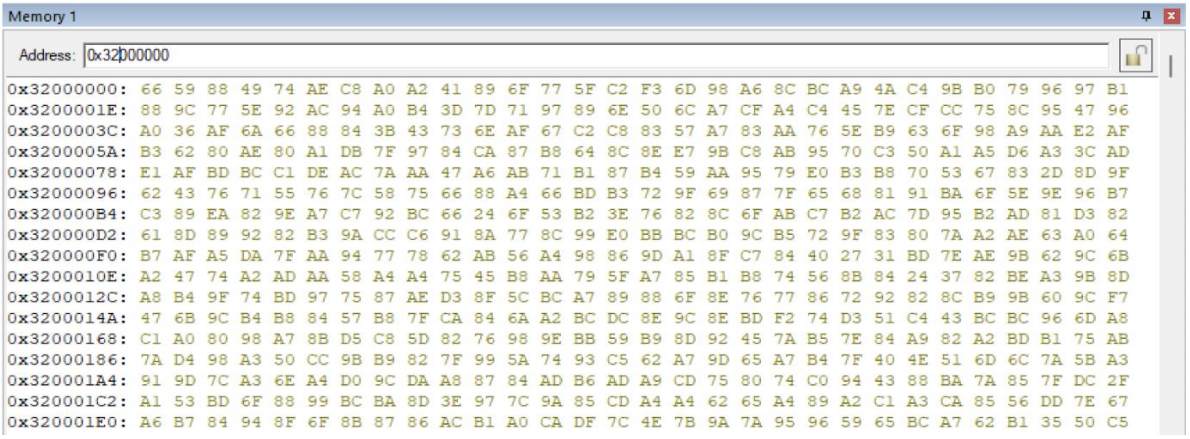
이 R11(종료 주소)보다 작은 지 비교한다. 작으면 'grayscale_loop' 로 다시 분기하여 루프를 반복한다. 무한 루프를 돌면서 프로그램 종료를 방지한다.

(3)-B. Memory map

Grayscale 변화를 수행하면 다음과 같은 결과가 나오게 된다.



이는 Function 2의 결과와 동일한 것을 볼 수 있다.



(4)-A. Function 4-3 : Function 4가 적용된 Binary ARM Code

```
69          STMFA R13!, {R1, R2, R3, R4}
70          MOV R6, #0x30000000 ; Memory address for loading 'R'
71          MOV R7, #0x31000000 ; Memory address for loading 'G'
72          MOV R8, #0x32000000 ; Memory address for loading 'B'
73          MOV R1, #0x25000000 ; Memory address for storing Binary bits
74          MOV R10, #0 ; Register storing binary bits (8 Binary Pixel = 1 Byte)
75          MOV R11, #7 ; Value for shift to combine binary bits
```

R6, R7, R8은 각각 R, G, B 값을 이미지에서 헤더를 분리한 픽셀 데이터에 접근하기 위한 주소 값을 가지고 있는 Register이다. Binary 비트를 저장할 메모리 주소를 R1에 저장한다. R10은 Binary 비트를 저장할 Register를 초기화하고 R11은 1-bit Binary 픽셀 8개를

하나의 바이트 주소에 저장하기 위해 Shift 연산에 사용할 값을 가지고 있는 Register이다.

```
77  BINARY_END
78          CMP R5, R8 ; If not R5 != R8 == 0x30096000
79          BNE BINARY ; Need more operation, Go to BINARY
80          B FUNC_END ; Else if R5 == R8 == 0x30096000, Function finished! Go to FUNC_END
81
82  BINARY
83          LDRB R2, [R6], #1 ; Load R of RGBA
84          LDRB R3, [R7], #1 ; Load G of RGBA
85          LDRB R4, [R8], #1 ; Load B of RGBA
86          MOV R9, #0 ; We will store ( max{R, G, B} + min{R, G, B} ) here
87          CMP R2, R3 ; Compare R with B
88          BGE R_IGT_G ; R >= G
89          B G_IGT_R ; R < G
```

R5와 R8(0x30096000)와 같은 지 비교하여 같지 않으면 BINARY로 분기하고 같으면 함수 종료로 분기한다.

'BINARY' 레이블에서 R값을 읽어와 R2에 저장하고, R6를 1byte 앞으로 이동한다. G, B 값도 동일하게 R3, R4에 저장하고 R7, R8을 1byte 앞으로 이동한다. Max{R, G, B}+min{R, G, B}를 저장할 R9를 초기화한다. R값과 G값을 비교하여 R>=G이면 R_IGT_G로 분기하고 R<G이면 G_IGT_R로 분기한다. 예를 들어 R>G>B로 가정하겠다.

```
91  R_IGT_G ; R >= G
92          CMP R2, R4
93          BGE R_IS_MAX ; R > G, B
94          B G_AND_B ; B > R > G
95
96  G_IGT_R ; G > R
97          CMP R3, R4
98          BGE G_IS_MAX ; G > R, B
99          B R_AND_B ; B > G > R
100
101 R_IS_MAX ; R > G, B
102         CMP R3, R4
103         BGE R_AND_B ; R > G > B
104         ADD R9, R9, R2 ; R > B > G
105         ADD R9, R9, R3
106         B CALCUL
107
108 G_IS_MAX ; G > R, B
109         CMP R2, R4
110         BGE G_AND_B ; G > R > B
111         ADD R9, R9, R2 ; G > B > R
112         ADD R9, R9, R3
113         B CALCUL
114
115 R_AND_B
116         ADD R9, R9, R2
117         ADD R9, R9, R4
118         B CALCUL
119
120 G_AND_B
121         ADD R9, R9, R3
122         ADD R9, R9, R4
123         B CALCUL
```

'R_IGT_G'로 분기 후, R과 B를 비교하여 R>=B이면 'R_IS_MAX'로 분기하고 B>R>G이면 'G_AND_B'로 분기한다. 'R_IS_MAX'로 분기 후 G와 B를 비교하여 G>=B이면 'R_AND_B'로

분기하고 R>B>G이면 R9에 R, G를 더한다. 'R_AND_B'로 분기 후 R9에 R과 B값을 더한 후 'CALCUL'로 분기한다.

```
125  CALCUL
126          LSR R9, R9, #1
127          CMP R9, #128
128          BGE BI_1 ; If R9 > 127, Current binary bit is 1
129          MOV R9, #0 ; Else if R9 <= 127, Current binary bit is 0
130          B BI_SHIFT
131
132  BI_1
133          MOV R9, #1 ; R9 > 127, Current binary bit is 1
```

R9를 오른쪽으로 1비트 shift하여 2로 나눈다. R9와 128을 비교하여 R9>127이면 BI_1로 분기하여 R9를 1로 설정한다. R<=127이면 0으로 설정한다. 그 후 'BI_SHIFT'로 분기한다.

'

```
135  BI_SHIFT
136          ADD R10, R10, R9, LSL R11
137          CMP R11, #0 ; If R11 == 0, R11 = 7
138          BEQ BI_STORE
139          SUB R11, R11, #1 ; R11 -= 1
140          B BINARY
141
142  BI_STORE
143          STRB R10, [R1], #1
144          MOV R10, #0
145          MOV R11, #7
146          B BINARY_END
147
148  FUNC_END
149          LDMFA R13!, {R1, R2, R3, R4}
150          END
```

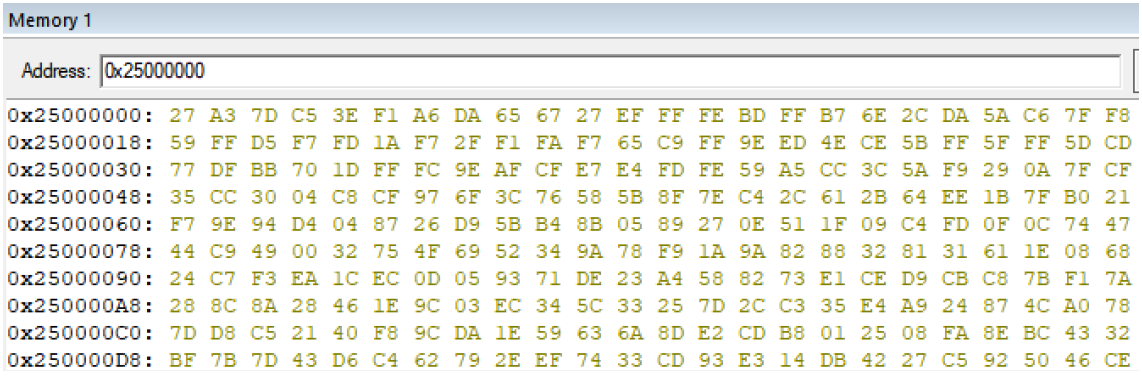
'BI_SHIFT'에서는 현재 계산된 이진 비트를 R10에 적절히 위치시키고 8비트가 완성될 때까지 반복한다. R9 왼쪽으로 R11만큼 shift한 값에 R10을 더한다. R11을 0과 비교하여 같으면 BI_STORE로 분기하고 같지 않으면 R11을 1 감소시켜 'BINARY'로 분기한다.

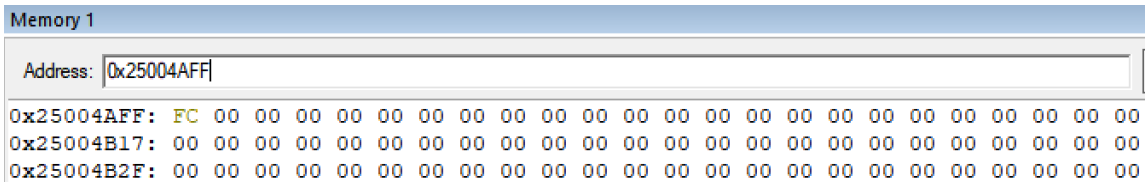

```
142  BI_STORE
143          STRB R10, [R1], #1
144          MOV R10, #0
145          MOV R11, #7
146          B BINARY_END
147
148  FUNC_END
149          LDMFA R13!, {R1, R2, R3, R4}
150          END
```

'BI_STORE'에서는 완성된 8비트를 메모리에 저장하고 다음 8비트를 위한 초기화를 수행한다. R10 값을 R1 주소에 저장하고 R1을 1 증가시킨다. R10을 0으로 초기화하고 R11도 7로 초기화하여 'BINARY_END'로 분기한다. 'FUNC_END'레이블에서는 스택에서 R1, R2, R3, R4를 복원한다.

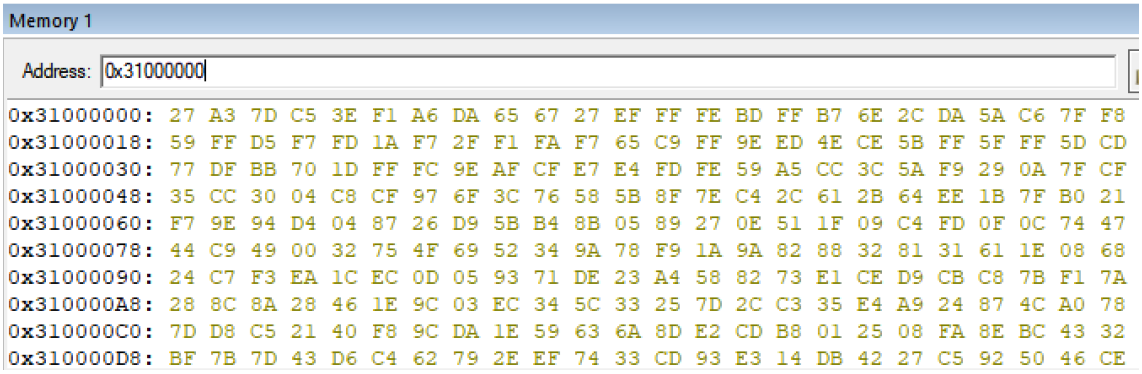
(4)-B. Memory map

Binary 변화를 수행하면 다음과 같은 결과가 나오게 된다.





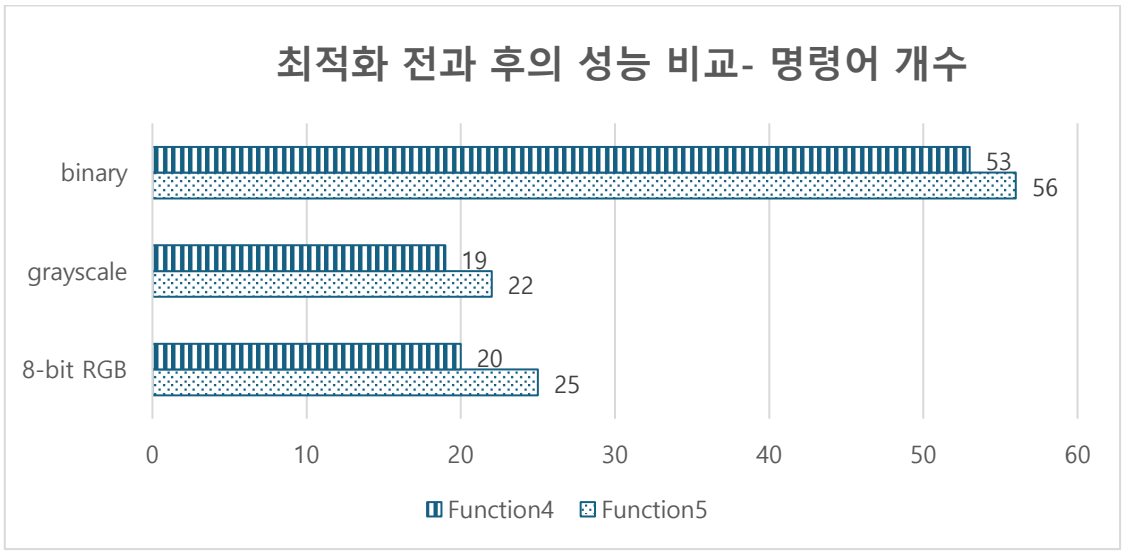
이는 Function 3와 비교했을 때 동일한 결과를 볼 수 있다.

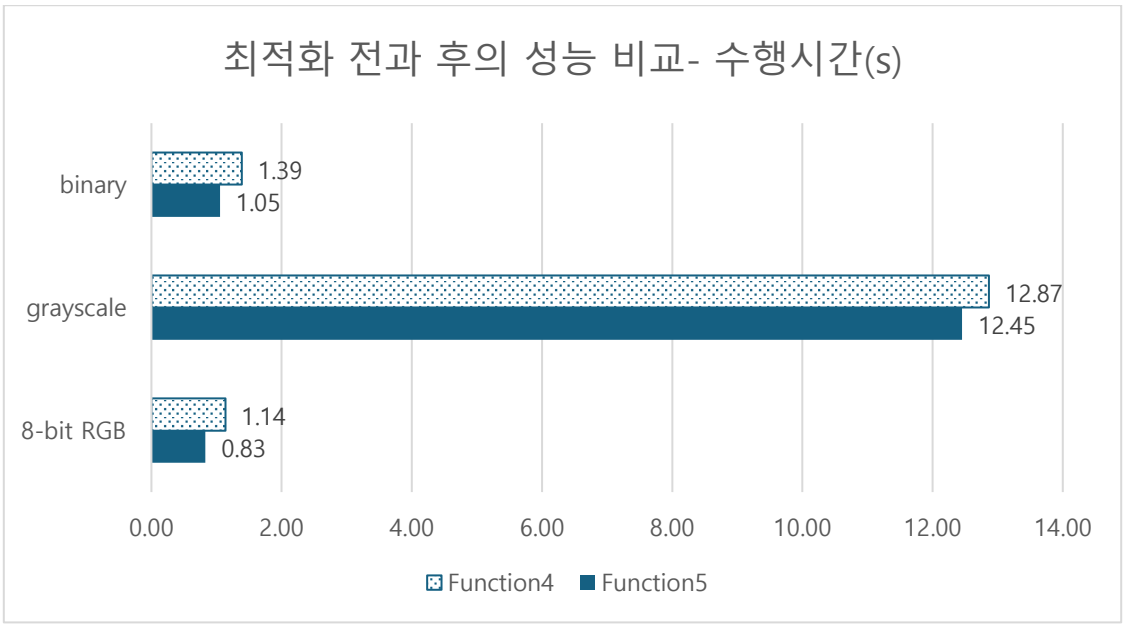


5. 기존 ARM Code와 최적화된 ARM code 사이의 성능 비교

(1) 성능 비교 설명

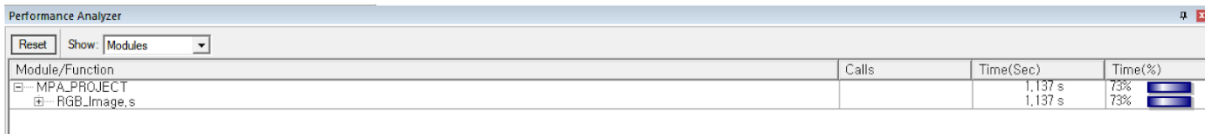
다음 표는 Function 4, Function 5의 명령어 개수를 비교한 자료이다. 8-bit RGB에서는 명령어 개수가 5개, Grayscale에서는 3개, Binary에서는 3개 증가했다. 이는 Memory Relocation을 수행하여 각 비트마다 불러오고 따로 저장해야 하기 때문이다. 수행시간(s)을 비교한 자료를 살펴보면 수행시간이 전반적으로 감소했다. 첫번째로 8-bit RGB에서 최적화 전에는 1.14s, 최적화 후에는 0.83s가 소모되었다. 이는 수행시간이 27.2% 감소했음을 알 수 있다. 또한 Binary에서 최적화 전에는 1.39s, 최적화 후에는 1.05s가 소모되었다. 이는 수행시간이 24.5% 감소했음을 알 수 있다.

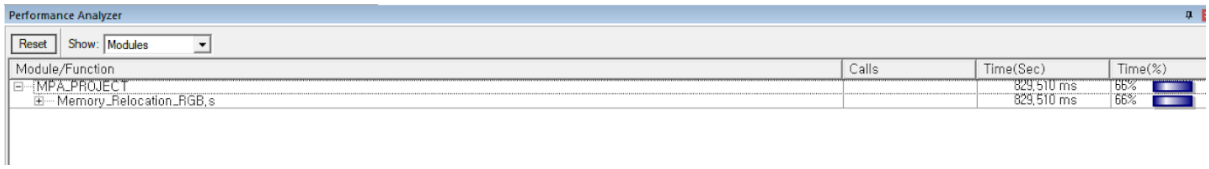




(2)-A. 8-bit RGB

다음은 Function 1과 Function 4-1에 대한 수행 시간 결과이다.

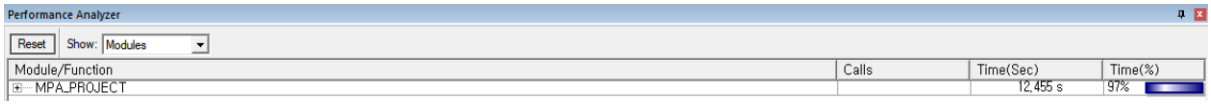




(2)-B. Grayscale

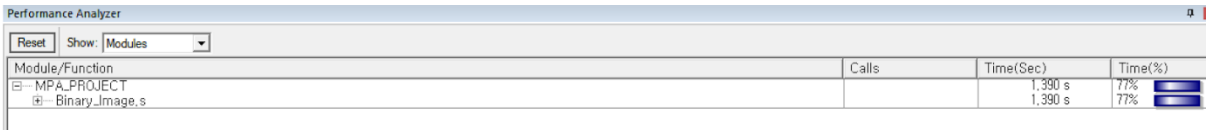
다음은 Function 2과 Function 4-2에 대한 수행 시간 결과이다.

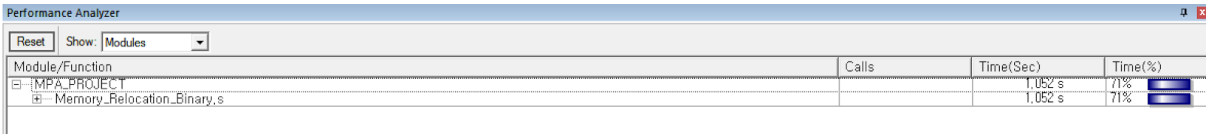




(3)-C. Binary

다음은 Function 3과 Function 4-3에 대한 수행 시간 결과이다.





6. ARM Code 추가 최적화

3가지 이미지 변환 방식에 대한 ARM Code를 작성해보며 명령어들의 동작 방식과 결과 등에 대해 알 수 있었고, 결과들이 Memory에 어떻게 저장되는지에 대해 확인할 수 있었다. 이후 기존에 작성한 ARM Code들의 성능을 확인해보며 추가적인 성능 향상을 위한 최적화를 계획하게 되었다. 여러 방법을 논의해본 결과 두 가지 측면에서 최적화를 진행하기로 하였다.

첫번째로 Memory Relocation 과정에서 최적화를 진행하였다. 먼저 기존에 Load되어 있던 Image의 Header와 Pixel 값을 Load 명령어를 통해 접근한 후 0으로 대체하는 동작을 생략하였다. 변환 방식에 따라 새로운 이미지를 만드는 과정에서 본 과정이 가독성 측면을 제외하면 필수가 아니라 생각하여 생략하였다. 다음으로, Header의 끝을 판단하는, 즉 "IDAT"를 확인하는 과정에서 판별 시 연산에서의 결과 값을 저장하는 Register를 단일화하고 주소 값을 다루는 Register 관련 명령어를 일부 생략함으로써 명령어의 개수를 대폭 줄였다. 주소 값을 다루는 Register 관련 명령어는 이미 저장된 Header 데이터를 확인할 때 Header 데이터가 저장된 주소 값만 복원, 값은 변하지 않았으므로 복원이 필요하지 않는 것을 확인하였고 이를 최적화에 포함하였다. 마지막으로, Pixel 데이터를 추출하는 과정에서 Alpha (투명도) 정보를 Skip하기 위해 사용하던 ADD 명령어를 B 값을 추출하는 과정에 주소 값을 추가로 증가시킴으로써 생략하였다. 이를 통해 두 가지 동작을 하나의 명령어에서 처리하면서 명령어의 개수를 줄일 수 있었다.

```
        AREA RESET, DATA, READONLY
        AREA OPT_RGB, CODE, READONLY
        ENTRY
        ; Instead of EXPORT, We add loading function code before main function code
        ; Initialization
        MOV R1, #0x40000000 ; Starting Address of Input (Image)
        MOV R2, #0x20000000 ; Memory Address of "Storing HEADER"
        MOV R3, #0x30000000 ; Memory Address of "Storing R"
        MOV R4, #0x31000000 ; Memory Address of "Storing G"
        MOV R5, #0x32000000 ; Memory Address of "Storing B"
        MOV R6, #0
        MOV R7, #0
HEADER ; Store bytes of HEADER from image
        LDRB R6, [R1], #1
        MOV R7, R6
        STRB R6, [R2], #1
        CMP R7, #0x54 ; Check that current byte is 'T'
        BNE HEADER ; If it is not 'T', keep loading
HEADER_END ; Check "IDAT"
        ; We will check 3 bytes before 'T'
        SUB R2, R2, #2
        MOV R10, #0
        LDRB R6, [R2], #-1 ; Check if it is 'A'
        CMP R6, #0x41
        ADDNE R10, R10, #1 ; If it is not 'A', mark at R10
        LDRB R6, [R2], #-1 ; Check if it is 'D'
        CMP R6, #0x44
        ADDNE R10, R10, #1 ; If it is not 'D', mark at R10
        LDRB R6, [R2], #-1 ; Check if it is 'I'
        CMP R6, #0x49
        ADDNE R10, R10, #1 ; If it is not 'I', mark at R10
        ; Restore to original state
        ADD R2, R2, #5
        CMP R10, #0 ; If R10 != 0, it is not "IDAT"
        BNE HEADER ; If it is "IDAT", terminate loading HEADER and start loading PIXEL
        MOV R7, #0 ; Initializing register to store PIXEL
PIXEL ; Store bytes of PIXEL from image
        LDRB R6, [R1], #1 ; Store R of RGBA
        STRB R6, [R3], #1
        LDRB R6, [R1], #1 ; Store G of RGBA
        STRB R6, [R4], #1
        LDRB R6, [R1], #2 ; Store B of RGBA and skip A of RGBA
        STRB R6, [R5], #1
        ADD R7, #4
        CMP R7, #6144000 ; 614400 = 960 * 160 * 4 (Bytes)
        BNE PIXEL
```

두번째로 함수 자체의 과정에서의 최적화를 진행하였다. 공통적으로 명령어 1개 또는 여러 개의 명령어가 모여 있는 Label 단위로 순서를 변경하여 명령어 실행을 보다 유연하게 하였고 이를 통해 불필요한 Branch 명령어를 감소시켰다. Branch 명령어를 감소시키는 것이 성능 향상에 있어 큰 비중을 차지하기 때문에 가장 먼저 진행하였다. 8-bit RGB 함수에서는 함수 종료를 위해 CMP 문을 통해 종료 조건을 확인하는 과정에서 3번의 확인 과정을 한 번으로 줄여 CMP, Branch 명령어를 감소시켰다. 함수 동작 구조 상 하나의 조건이 만족되면 세 개의 조건이 모두 만족되는 것과 동일하므로 조건 판별 또한 한 번만 진행해도 된다고 판단하였다. Grayscale 함수에서는 Grayscale 값을 계산하는 과정을 R, G, B 값의 평균으로 계산하는 방식에서 최대값과 최소값의 평균으로 계산하는 방식으로 변경하였고, 함수 동작 횟수를 종료 주소 값을 직접 지정하여 실행하는 것이 아닌 Pixel 데이터가 있는 만큼만 실행함으로써 최적화를 대폭 진행하였다.

```
48          ; Here is starting point of Function 1, Convert to RGB
49
50          MOV R7, #0x30000000 ; Memory address of R
51          MOV R8, #0x31000000 ; Memory address of G
52          MOV R9, #0x32000000 ; Memory address of B
53          MOV R11, #0x25000000 ; Memory address of 8-bit RGB
54  RGB
55          MOV R10, #0 ; We will store RGB here
56          LDRB R6, [R7], #1 ; Load R of RGBA
57          AND R6, R6, #0xE0 ; Extract R[7:5]
58          ADD R10, R10, R6
59          LDRB R6, [R8], #1 ; Load G of RGBA
60          AND R6, R6, #0xE0 ; Extract G[7:5]
61          LSR R6, R6, #3 ; Shift to make RGB
62          ADD R10, R10, R6
63          LDRB R6, [R9], #1 ; Load B of RGBA
64          AND R6, R6, #0xC0 ; Extract B[7:6]
65          LSR R6, R6, #6 ; Shift to make RGB
66          ADD R10, R10, R6
67          STRB R10, [R11], #1 ; Store RGB Byte
68          CMP R9, R5 ; If R9 == R5 , Converting is over
69          BNE RGB
70          END
```

< 8-bit RGB 최적화 Code >

```
51  ; ---------- Here is starting point of Function 2, Converting to Grayscale ----------
52
53          STMFA R13!, {R1, R2, R3, R4} ; Store original state at stack to extract PIXEL
54          MOV R6, #0x30000000 ; Memory address for loading 'R'
55          MOV R7, #0x31000000 ; Memory address for loading 'G'
56          MOV R8, #0x32000000 ; Memory address for loading 'B'
57          MOV R1, #0x25000000 ; Memory address for storing Binary bits
58  GRAYSCALE
59          LDRB R2, [R6], #1 ; Load R of RGBA
60          LDRB R3, [R7], #1 ; Load G of RGBA
61          LDRB R4, [R8], #1 ; Load B of RGBA
62          MOV R9, #0 ; We will store ( max(R, G, B) + min(R, G, B) ) here
63          CMP R2, R3 ; Compare R with B
64          BLS G_IGT_R ; G > R
65
66  R_IGT_G ; R >= G
67          CMP R2, R4
68          BGE R_IS_MAX ; R > G, B
69          B G_AND_B ; B > R > G
70
71  G_IGT_R ; G > R
72          CMP R3, R4
73          BGE G_IS_MAX ; G > R, B
74          B R_AND_B ; B > G > R
75
76  R_IS_MAX ; R > G, B
77          CMP R3, R4
78          BGE R_AND_B
79          B R_AND_G
80
81  G_IS_MAX ; G > R, B
82          CMP R2, R4
83          BGE G_AND_B
84          B R_AND_G
85
86  R_AND_B ; R > G > B
87          ADD R9, R9, R2
88          ADD R9, R9, R4
89          B CALCUL
90
```

```
R_AND_G ; R > B > G
        ADD R9, R9, R2
        ADD R9, R9, R3
        B CALCUL

G_AND_B ; G > R > B
        ADD R9, R9, R3
        ADD R9, R9, R4

CALCUL
        LSR R9, R9, #1
        STRB R9, [R1], #1
        CMP R5, R8 ; If not R5 != R8 == 0x30096000
        BNE GRAYSCALE ; If R5 != R8 == 0x30096000, Function not finished! Go to GRAYSCALE
        LDMFA R13!, {R1, R2, R3, R4} ; Load original state
        END
```

< Grayscale 최적화 Code >

```
; ---------- Here is starting point of Function 3, Converting to Binary ----------

        STMFA R13!, {R1, R2, R3, R4} ; Store original state at stack to extract PIXEL
        MOV R6, #0x30000000 ; Memory address for loading 'R'
        MOV R7, #0x31000000 ; Memory address for loading 'G'
        MOV R8, #0x32000000 ; Memory address for loading 'B'
        MOV R1, #0x25000000 ; Memory address for storing Binary bits
        MOV R10, #0 ; Register storing binary bits (8 Binary Pixel = 1 Byte)
        MOV R11, #7 ; Value for shift to combine binary bits

BINARY
        LDRB R2, [R6], #1 ; Load R of RGBA
        LDRB R3, [R7], #1 ; Load G of RGBA
        LDRB R4, [R8], #1 ; Load B of RGBA
        MOV R9, #0 ; We will store ( max(R, G, B) + min(R, G, B) ) here
        CMP R2, R3 ; Compare R with B
        BGE R_IGT_G ; R >= G
        B G_IGT_R ; R < G

R_IGT_G ; R >= G
        CMP R2, R4
        BGE R_IS_MAX ; R > G, B
        B G_AND_B ; B > R > G

G_IGT_R ; G > R
        CMP R3, R4
        BGE G_IS_MAX ; G > R, B
        B R_AND_B ; B > G > R

R_IS_MAX ; R > G, B
        CMP R3, R4
        BGE R_AND_B ; R > G > B
        ADD R9, R9, R2 ; R > B > G
        ADD R9, R9, R3
        B CALCUL
```

```
G_IS_MAX ; G > R, B
        CMP R2, R4
        BGE G_AND_B ; G > R > B
        ADD R9, R9, R2 ; G > B > R
        ADD R9, R9, R3
        B CALCUL

R_AND_B
        ADD R9, R9, R2
        ADD R9, R9, R4
        B CALCUL

G_AND_B
        ADD R9, R9, R3
        ADD R9, R9, R4

CALCUL
        LSR R9, R9, #1
        CMP R9, #128
        BGE BI_1
        MOV R9, #0 ; Else if R9 <= 127, Current binary bit is 0

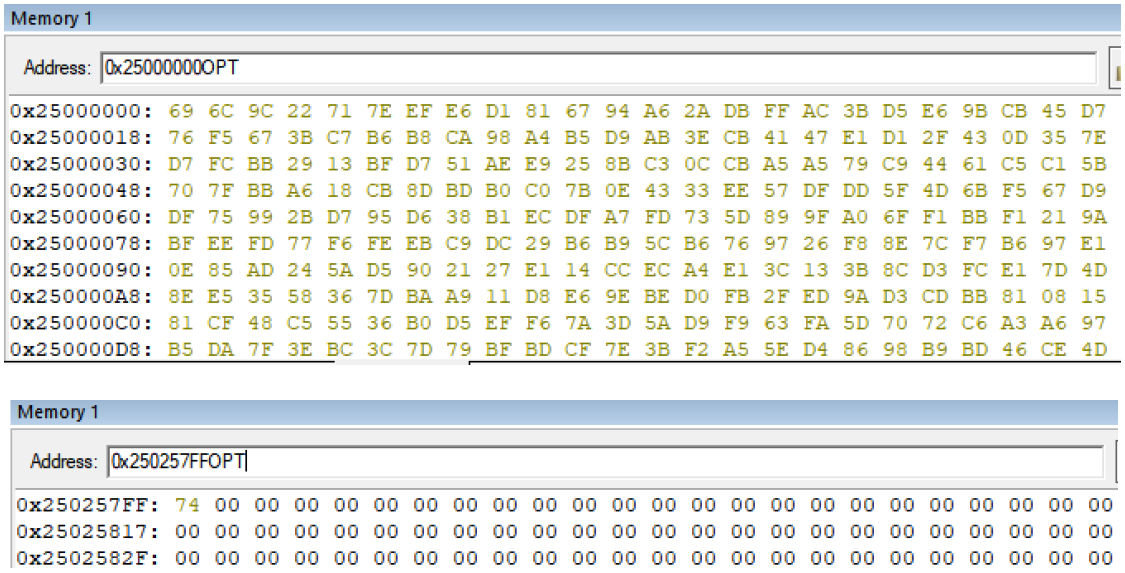
BI_SHIFT
        ADD R10, R10, R9, LSL R11 ; Shift to store 8 bits to 1 byte
        CMP R11, #0 ; If R11 == 0, it is last bit of 8 Binary bits
        BEQ BI_STORE
        SUB R11, R11, #1 ; Else if R11 != 0, R11 -= 1
        B BINARY

BI_1
        MOV R9, #1 ; R9 > 127, Current binary bit is 1
        B BI_SHIFT


BI_STORE ; 8 Binary bits loaded, Time to store!
        STRB R10, [R1], #1
        MOV R10, #0 ; Initializing to continue loading and storing
        MOV R11, #7

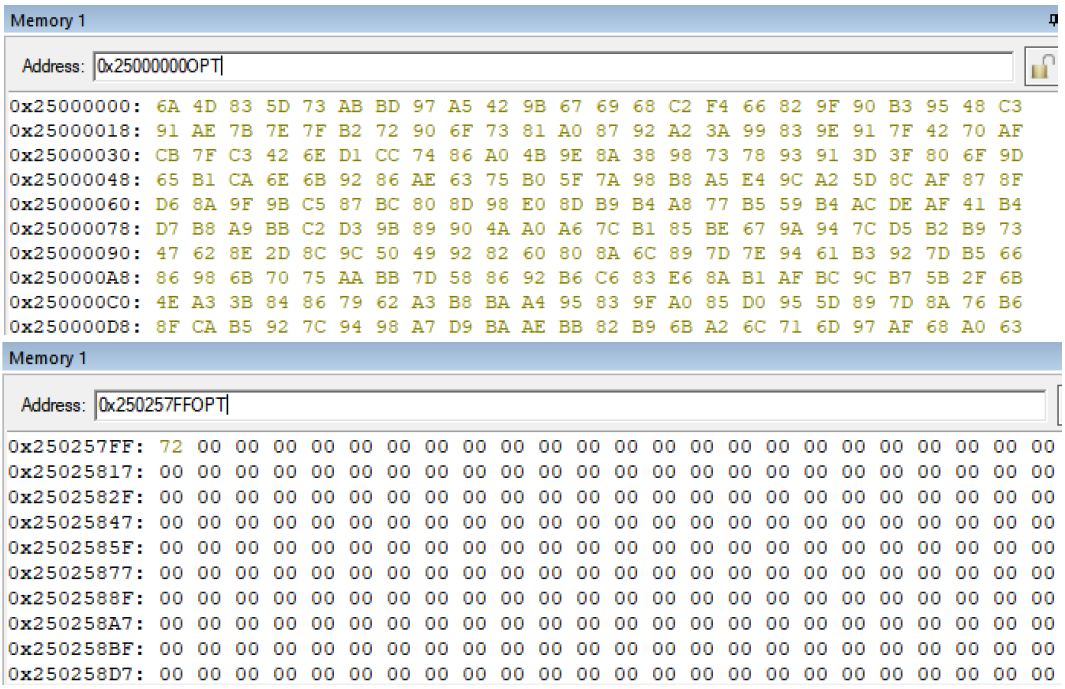
BINARY_END
        CMP R5, R8 ; If not R5 != R8 == 0x30096000
        BNE BINARY ; If R5 != R8 == 0x30096000, Function not finished! Go to BINARY
        LDMFA R13!, {R1, R2, R3, R4} ; Load original state
        END
```

< Binary 최적화 Code >



< 추가 최적화 8-bit RGB 함수의 Memory Map >

주소에 쓰여진 OPT는 이미지 상 구분을 위해 써놓은 것이다. (의미 X)



< 추가 최적화 Grayscale 함수의 Memory Map >

Memory 1

Address: 0x25000000OPT

```
0x25000000: 27 A3 7D C5 3E F1 A6 DA 65 67 27 EF FF FE BD FF B7 6E 2C DA 5A C6 7F F8
0x25000018: 59 FF D5 F7 FD 1A F7 2F F1 FA F7 65 C9 FF 9E ED 4E CE 5B FF 5F FF 5D CD
0x25000030: 77 DF BB 70 1D FF FC 9E AF CF E7 E4 FD FE 59 A5 CC 3C 5A F9 29 0A 7F CF
0x25000048: 35 CC 30 04 C8 CF 97 6F 3C 76 58 5B 8F 7E C4 2C 61 2B 64 EE 1B 7F B0 21
0x25000060: F7 9E 94 D4 04 87 26 D9 5B B4 8B 05 89 27 0E 51 1F 09 C4 FD 0F 0C 74 47
0x25000078: 44 C9 49 00 32 75 4F 69 52 34 9A 78 F9 1A 9A 82 88 32 81 31 61 1E 08 68
0x25000090: 24 C7 F3 EA 1C EC 0D 05 93 71 DE 23 A4 58 82 73 E1 CE D9 CB C8 7B F1 7A
0x250000A8: 28 8C 8A 28 46 1E 9C 03 EC 34 5C 33 25 7D 2C C3 35 E4 A9 24 87 4C A0 78
0x250000C0: 7D D8 C5 21 40 F8 9C DA 1E 59 63 6A 8D E2 CD B8 01 25 08 FA 8E BC 43 32
0x250000D8: BF 7B 7D 43 D6 C4 62 79 2E EF 74 33 CD 93 E3 14 DB 42 27 C5 92 50 46 CE
```

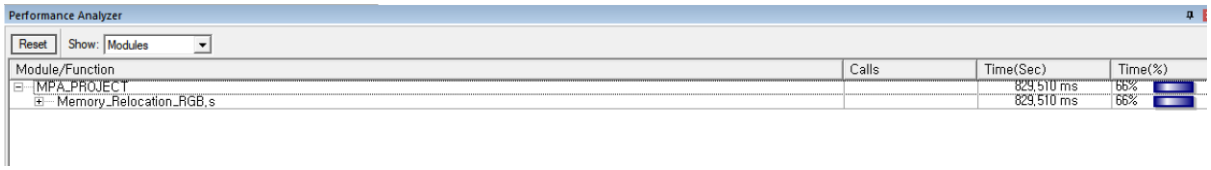
Memory 1

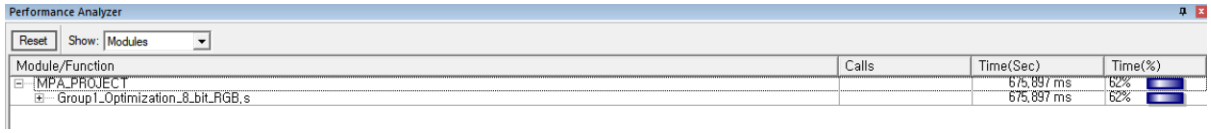
Address: 0x25004AFFOPT

```
0x25004AFF: FC 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x25004B17: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x25004B2F: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

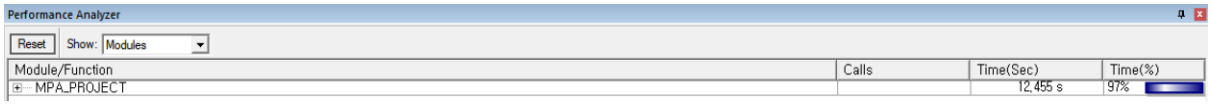
< 추가 최적화 Binary 함수의 Memory Map >

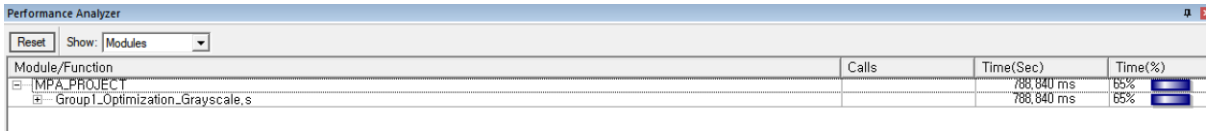
위 방식대로 최적화를 진행한 후 성능을 비교해보았다.



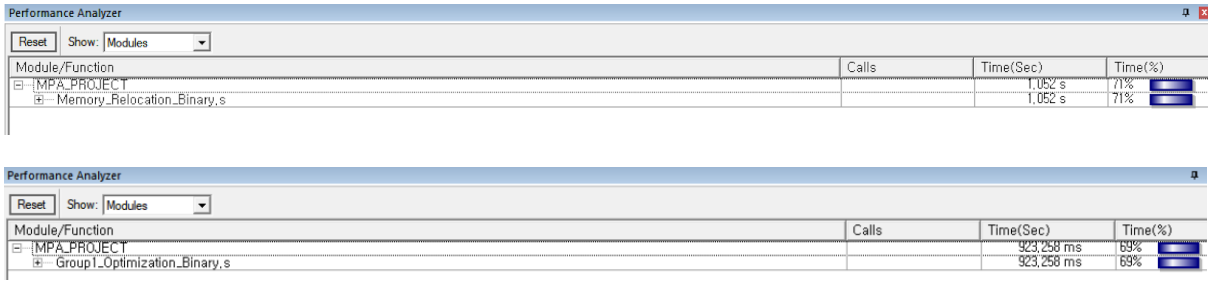


< 8-bit RGB 함수 최적화 후 >





< Grayscale 함수 최적화 후 >



< **Binary 함수 최적화 후** >

최적화 진행 결과 8-bit RGB와 Binary 함수에는 10% 이상의 Run-Time 감소를 보였고, Grayscale 함수에서는 큰 감소를 볼 수 있었다. 명령어의 개수 또한 8-bit RGB와 Binary 함수에서는 19개, Grayscale에서는 2개 감소시킬 수 있었다.

7. 실제 알고리즘에 기반한 결과 (이미지) 도출

PNG 파일 포맷에서 사용되는 압축 방식은 LZ77 알고리즘과 함께 Huffman 코딩을 기반으로 하는 DEFLATE 압축 알고리즘을 사용해서 PNG 파일에서 볼 수 있는 HEX 값은 일반적으로 압축된 데이터이며, 직접적으로는 원본 이미지의 RGBA 값이 아니다. 따라서 프로젝트를 하기 위해 받은 2024S_MA_image_rgba.hex 파일에서 Header 부분을 제외하고 '78' 값부터 시작하는 Pixel Data의 RGBA는 실제 RGBA값이 아니다.

PNG 파일 내부의 실제 RGBA 값을 알고 싶으면 파일을 Decode 하거나 압축 해제 과정을 거쳐야 하는데 본 프로젝트에서는 Python의 "Pillow" 라이브러리를 이용하여 PNG 파일을 열고 실제 RGBA 값을 가져온 후 Header 부분이나 IDAT, IEND등을 제외한 RGBA 값만 나오는 새로운 RGBA_output.hex 파일로 만들었다. (메모리 주소는 0x20000000부터 값 저장)

```python
from intelhex import IntelHex
from PIL import Image
import numpy as np

def extract_rgba_to_hex(input_image_file, output_hex_file):
    # Load the image using Pillow
    image = Image.open(input_image_file)
    rgba_data = np.array(image)

    ih = IntelHex()

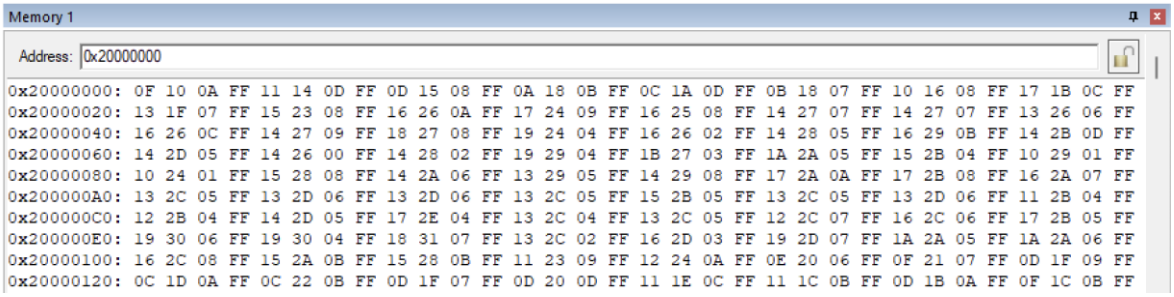
    # Flatten the RGBA data to a 1D array and write to IntelHex object
    address = 0x20000000
    for pixel in rgba_data.reshape(-1, 4):
        r, g, b, a = pixel
        ih[address] = r
        ih[address + 1] = g
        ih[address + 2] = b
        ih[address + 3] = a
        address += 4

    # Save the IntelHex object to a file
    ih.write_hex_file(output_hex_file)

# Input and output files
input_image_file = '2024S_MA_image_rgba.png'  # Update with your image file
output_hex_file = 'RGBA_output.hex'

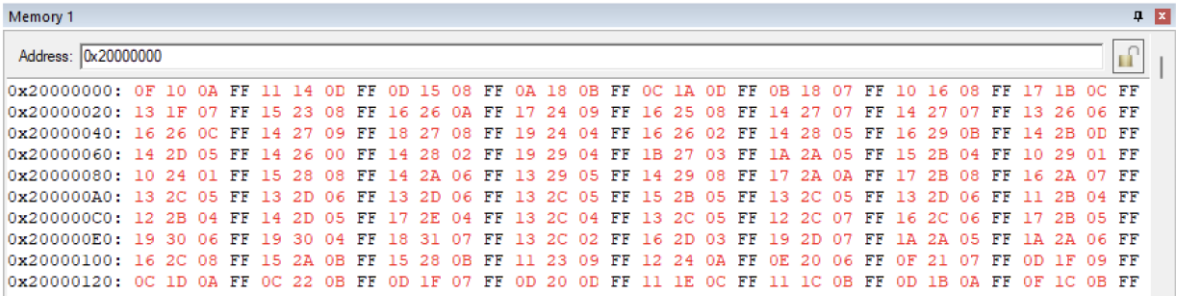
# Extract RGBA data from the image and save to a new HEX file
extract_rgba_to_hex(input_image_file, output_hex_file)
```

< **압축 해제 후 RGBA값만 가져오는 새로운 .hex 파일 생성** >

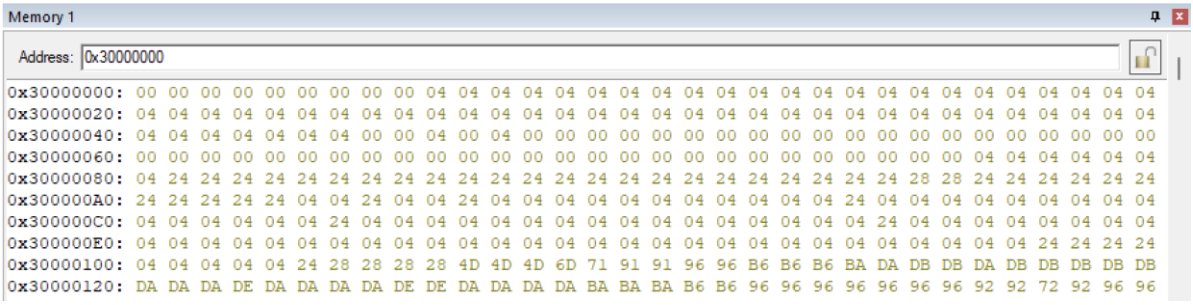


< **압축 해제한 실제 RGBA 값** >

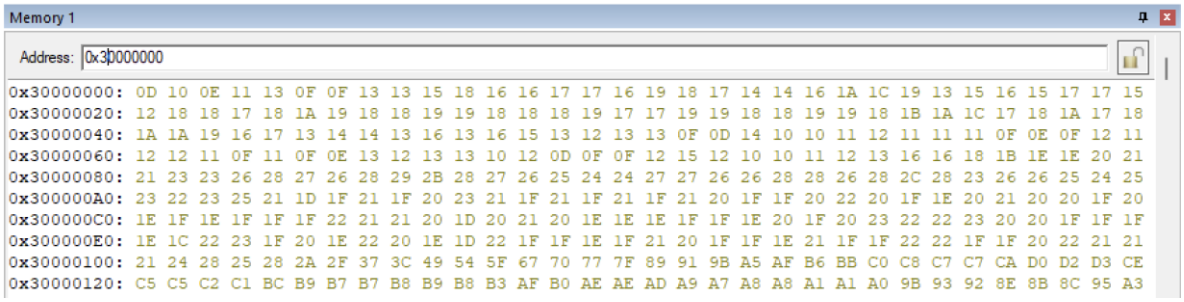
이후 각 8-bit RGB, Grayscale, Binary 코드를 새로 만든 RGBA_output.hex의 메모리 주소에 맞게 변환하였다.



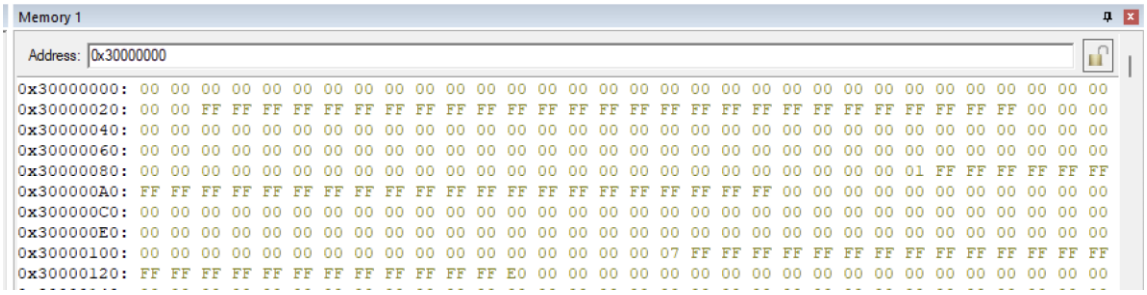
< **0x20000000에 있는 RGB 값을 이용, 0x30000000으로 함수를 진행한 값 넣는 과정** >



< **8-bit RGB 변환 값** >



< **Grayscale 변환 값** >



< **Binary 변환 값** >

이후 각각 0x30000000번지부터 저장되어 있는 값을 새로운 000_Image_Output.hex로 Save하여 변환한 값을 새로운 hex로 만들고, 변환된 결과를 PNG로 확인하기 위하여 Python으로 변환한 hex파일을 불러와서 1byte로 되어 있는 값들을 각각 함수에 맞게 RGB 3byte로 변환한 후 A에 255값을 추가하여 한 픽셀 당 4byte인 PNG 형태로 변환하였다.



< 추출한 Image 사진 (**왼쪽부터 8-bit RGB, Grayscale, Binary**) >

Memory Optimization에서 명령어를 최적화한 후 해당 결과를 이용하여도 메모리 byte 값과 이미지가 똑같이 나오는 것을 확인했다.

Ⅲ. 결론

1. 프로젝트 수행에 대한 고찰 (Troubleshooting)

프로젝트 초기에는 팀원들과 보고서 항목대로 역할을 나누었지만 프로젝트를 진행하면서 수행함에 있어 어려움을 느껴 Function Coding 파트 단위로 나누고, 그 파트를 담당한 인원끼리 보고서 작성을 담당하기로 변경하였다. 그로 인해 수행 내용에 대한 이해도가 좋아졌고 프로젝트 진행 속도 또한 빨라지게 되었다. 또한, Grayscale 구현 부분에서 (R + G + B) / 3 방식과 {MAX(R, G, B) + MIN(R, G, B)} / 2 방식 모두 사용해보았다. 3으로 나누는 방식을 진행해보았는데 ARM 코드에서는 Divide 기능을 지원하지 않아서 Shift를 통해 복잡하게 구현해야 했다. 따라서 Code 작성 시 별도의 함수를 이용하여 함수 안에서의 Loop를 돌며 3씩 뺄셈하고 몫을 1씩 더해가면서 함수가 끝나면 최종 몫이 새로운 Register에 저장되는 방식을 이용하였다. 하지만 이 방식을 사용하니 한 번의 몫을 계산할 때 Loop를 너무 많이 돌게 되어 성능적인 면에서 문제가 있었다. 또한 3으로 나누는 방식은 1byte 단위로 나오는 결과 값이 실제 계산 값과 다른 값이 나오는 경우가 존재하는 것을 확인하였다. 따라서 함수 작성 및 Memory Relocation을 이용한 함수 작성 시 3으로 빼는 기존 방식에서 Optimization에서는 RGB 중 Max와 Min을 2로 나누는 방법을 선택하였다. 이를 통해 명령어 최적화 부분에서 원활하게 시간이 단축되는 것을 확인하였다.

이러한 문제점들을 팀원 간 원활한 의사소통을 통해 해결 해봄으로써 문제 해결에 있어 원활한 방식이 무엇이고 실제로는 어떻게 행동해야 하는지에 대해서도 고찰해볼 수 있었다.

2. 프로젝트 결과와 느낀 점

본 프로젝트를 통해 ARM Code에 대한 이해와 이를 통해 Image의 Pixel 데이터를 직접 다루는 Code로 구현해보며 Memory 주소에 대한 이해와 실제로 다루는 법에 대해 공부하는 시간을 보내게 되었다. 또한, 추가적인 최적화를 진행하며 성능과 관련된 ARM Code 최적화는 어떤 부분에서 진행하면 되는지에 대해 고찰해보고 적용해보며 ARM Code와 Memory에 대한 심화 학습 또한 진행할 수 있었다. 마지막으로 실제 이미지 결과를 추출함으로써 변환 이미지 각각이 어떤 특징을 가지고 있고 해당 이미지들은 데이터 상으로 어떤 방식으로 어떤 특징을 가지고 생성되는지 알 수 있었다.

[보고서에 작성한 내용 중 사용한 자료 출처]

ARM 공식 판매처 https://www.arm.co.kr/